

Reporte Técnico Completo

Análisis Infraestructura

» Descargas » infra-test »

Nombre	Fecha de modificación	Tipo	Tamaño
 .github	1/04/2026 9:15 p. m.	Carpeta de archivos	
 app	1/04/2026 9:15 p. m.	Carpeta de archivos	
 nginx	1/04/2026 9:15 p. m.	Carpeta de archivos	
 .env	1/04/2026 4:23 a. m.	Archivo ENV	1 KB
 docker-compose.yml	1/04/2026 10:21 p. m.	Archivo YML	1 KB
 README.md	1/04/2026 10:35 p. m.	Archivo MD	1 KB

Por el contenido nos podemos dar de cuenta que es una aplicacion en Python (por el archivo ./app/app.py) que funciona dentro de un contenedor Docker (por el archivo docker-compose.yml) el cual a su vez necesita del servicio web Nginx para funcionar (por el contenido del directorio ./nginx)

Implementación y preparación del entorno (sobre Docker en Debian 12 + herramientas adicionales)

Aplicar updates e instalación de docker y docker compose

```
deivi@debian:~$ sudo apt update && sudo apt upgrade -y
```

```
deivi@debian:~$ sudo apt install -y docker.io docker-compose
```

```
deivi@debian:~$ sudo systemctl enable --now docker
Synchronizing state of docker.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable docker
```

```

deivi@debian:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-04-12 18:31:01 -05; 44s ago
 TriggeredBy: ● docker.socket
    Docs: https://docs.docker.com
   Main PID: 55440 (dockerd)
     Tasks: 10
    Memory: 33.5M
       CPU: 346ms
    CGroup: /system.slice/docker.service
           └─55440 /usr/sbin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

abr 12 18:31:00 debian dockerd[55440]: time="2026-04-12T18:31:00.494533413-05:00" level=info msg="[core] Subch>
abr 12 18:31:00 debian dockerd[55440]: time="2026-04-12T18:31:00.494556933-05:00" level=info msg="[core] Chann>
abr 12 18:31:00 debian dockerd[55440]: time="2026-04-12T18:31:00.667209535-05:00" level=info msg="[graphdriver>
abr 12 18:31:00 debian dockerd[55440]: time="2026-04-12T18:31:00.678947485-05:00" level=info msg="Loading cont>
abr 12 18:31:01 debian dockerd[55440]: time="2026-04-12T18:31:01.156399467-05:00" level=info msg="Default brid>
abr 12 18:31:01 debian dockerd[55440]: time="2026-04-12T18:31:01.229532807-05:00" level=info msg="Loading cont>
abr 12 18:31:01 debian dockerd[55440]: time="2026-04-12T18:31:01.246409127-05:00" level=info msg="Docker daemo>
lines 1-19...skipping...

```

```

deivi@debian:~$ sudo usermod -aG docker $USER
deivi@debian:~$ groups
deivi cdrom floppy audio dip video plugdev users kvm netdev docker

```

```

deivi@debian:~$ init 6

```

Conexión por SSH y SFTP para subir el archivo.rar y administración del sistema

4. 200.10.20.4(deivi)						
/home/deivi/						
Name	Size (KB)	Last modified	Owner	Group	Access	Size (Bytes)
..						0
.cache		2025-01-22 20:53	deivi	deivi	drwxr-xr-x	4096
.config		2025-01-22 20:53	deivi	deivi	drwx-----	4096
.docker		2025-01-29 18:44	deivi	deivi	drwx-----	4096
.gnupg		2024-11-18 20:48	deivi	deivi	drwx-----	4096
.local		2024-11-18 20:48	deivi	deivi	drwx-----	4096
.mozilla		2026-04-10 13:29	deivi	deivi	drwx-----	4096
.pki		2025-01-22 20:53	deivi	deivi	drwx-----	4096
Descargas		2025-01-22 20:52	deivi	deivi	drwxr-xr-x	4096
Documentos		2024-11-18 20:48	deivi	deivi	drwxr-xr-x	4096
Escritorio		2024-11-18 20:48	deivi	deivi	drwxr-xr-x	4096
Imágenes		2024-11-18 20:48	deivi	deivi	drwxr-xr-x	4096
lab-devops		2026-04-10 19:14	deivi	deivi	drwxr-xr-x	4096
my-docker-app		2025-01-29 18:44	deivi	deivi	drwxr-xr-x	4096
Música		2024-11-18 20:48	deivi	deivi	drwxr-xr-x	4096
Plantillas		2024-11-18 20:48	deivi	deivi	drwxr-xr-x	4096
Público		2024-11-18 20:48	deivi	deivi	drwxr-xr-x	4096
Videos		2024-11-18 20:48	deivi	deivi	drwxr-xr-x	4096
.bash_history	8	2026-04-10 13:29	deivi	deivi	-rw-----	8793
.bash_logout	1	2024-11-13 20:08	deivi	deivi	-rw-r--r--	220
.bashrc	3	2024-11-13 20:08	deivi	deivi	-rw-r--r--	3526
.dmrc	1	2024-11-18 20:48	deivi	deivi	-rw-r--r--	35
.gitconfig	1	2026-04-10 15:54	deivi	deivi	-rw-r--r--	95
.ICEauthority	0	2024-11-18 20:48	deivi	deivi	-rw-----	0
.lessht	1	2026-04-10 18:10	deivi	deivi	-rw-----	20
.profile	1	2024-11-13 20:08	deivi	deivi	-rw-r--r--	807
.Xauthority	1	2026-04-10 14:55	deivi	deivi	-rw-----	155
.xsession-errors	10	2026-04-10 20:22	deivi	deivi	-rw-----	11002
.xsession-errors.old	3	2026-04-10 13:29	deivi	deivi	-rw-----	3200
infra-test.rar	1	2026-04-10 20:34	deivi	deivi	-rw-r--r--	1765

Instalación Unrar para Descomprimir el directorio

```
deivi@debian:~$ sudo apt install -y unrar-free
```

```
deivi@debian:~$ mkdir infra-test
deivi@debian:~$ unrar -x infra-test.rar infra-test

unrar-free 0.1.3 Copyright (C) 2004 Ben Asselstine, Jeroen Dekkers

Extracting from /home/deivi/infra-test.rar

Extracting .github/workflows/deploy.yml OK
Extracting app/app.py OK
Extracting app/Dockerfile OK
Extracting app/requirements.txt OK
Extracting nginx/default.conf OK
Extracting .env OK
Extracting docker-compose.yml OK
Extracting README.md OK
All OK
deivi@debian:~$ ls
Descargas Escritorio infra-test Música Plantillas Vídeos
Documentos Imágenes infra-test.rar my-docker-app Público

deivi@debian:~/infra-test$ ls
app docker-compose.yml nginx README.md
```

Tarea 1: Levantamiento del entorno

Ya instalado Docker vamos a proceder a levantar el entorno para revisar que errores tenemos que nos ayuden a solucionar el problema reportado

```
deivi@debian:~/infra-test$ docker-compose up --build
ERROR: Service 'nginx' depends on service 'backend' which is undefined.
```

Lo primero que vemos es que tenemos un error en el servicio nginx donde nos indica que el servicio backend no está definido.

Lo primero es revisar el contenido del archivo de configuración del servicio web `./nginx/default.conf`

```
server {  
    listen 80;  
  
    location / {  
        proxy_pass http://backend:8000;  
    }  
}
```

Podemos deducir que aca se nos indica como conectarnos por http al servidor web, donde el nombre de host donde realizaremos la conexión es "backend" y el puerto de escucha de la aplicacion es el 8000

Tarea 2, 3 y 4: Identificación y Solución primer error

A continuación vamos a revisar el archivo de docker-compose.yml que es el que permite levantar el contenedor a ver si encontramos algún error

```
version: '3.8'  
  
services:  
  web:   
    build: ./app  
    container_name: app  
    environment:  
      - ENVIRONMENT=production  
    expose:  
      - "5000" 
```

Podemos evidenciar 1 posible error y es que el servicio aca se llama "web" y en nginx se llama "backend" asi que vamos a corregir el nombre y volveremos a levantar el entorno.

```
version: '3.8'

services:
  ## web:
  backend:
    build: ./app
    container_name: app
    environment:
      - ENVIRONMENT=production
    expose:
      - "5000"
```

Como podemos ver ya levanto el entorno y comenzó a configurar los contenedores

```
deivi@debian:~/infra-test$ docker-compose up --build
Creating network "infra-test_default" with the default driver
Building backend
Sending build context to Docker daemon  4.096kB
Step 1/5 : FROM python:3.10
3.10: Pulling from library/python
```

```
deivi@debian:~/infra-test$ docker-compose up --build
Building backend
Sending build context to Docker daemon  5.632kB
Step 1/5 : FROM python:3.10
---> 738f70a7db01
Step 2/5 : WORKDIR /app
---> Using cache
---> 5d3f8adcffa5
Step 3/5 : COPY . .
---> d5399f53274d
Step 4/5 : RUN pip install -r requirements.txt
---> Running in a16f2ebf9698
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl (103 kB)
  103.4/103.4 kB 3.6 MB/s eta 0:00:00
Collecting click>=8.1.3
  Downloading click-8.3.2-py3-none-any.whl (108 kB)
```

108.4/108.4 kB 8.2 MB/s eta 0:00:00

Collecting itsdangerous>=2.2.0

Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)

Collecting werkzeug>=3.1.0

Downloading werkzeug-3.1.8-py3-none-any.whl (226 kB)

226.5/226.5 kB 9.0 MB/s eta 0:00:00

Collecting jinja2>=3.1.2

Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)

134.9/134.9 kB 29.1 MB/s eta 0:00:00

Collecting blinker>=1.9.0

Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)

Collecting markupsafe>=2.1.1

Downloading markupsafe-3.0.3-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (20 kB)

Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, flask

Successfully installed blinker-1.9.0 click-8.3.2 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.8

WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>

[notice] A new release of pip is available: 23.0.1 -> 26.0.1

[notice] To update, run: pip install --upgrade pip

Removing intermediate container a16f2ebf9698

---> 5606b0942acf

Step 5/5 : CMD ["python", "app.py"]

---> Running in ab01c5486dcf

Removing intermediate container ab01c5486dcf

---> ce43e8c78d6a

Successfully built ce43e8c78d6a

Successfully tagged infra-test_backend:latest

Recreating app ... done

Recreating proxy ... done

Attaching to app, proxy

proxy | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration

proxy | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/

app | * Serving Flask app 'app'

app | * Debug mode: off

proxy | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh

proxy | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf

app | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

app | * Running on all addresses (0.0.0.0)

app | * Running on http://127.0.0.1:5000

app | * Running on http://172.19.0.2:5000

app | Press CTRL+C to quit

```
proxy | 10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf differs from the packaged version
proxy | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
proxy | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
proxy | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
proxy | /docker-entrypoint.sh: Configuration complete; ready for start up
proxy | 2026/04/13 00:30:19 [notice] 1#1: using the "epoll" event method
proxy | 2026/04/13 00:30:19 [notice] 1#1: nginx/1.29.8
proxy | 2026/04/13 00:30:19 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
proxy | 2026/04/13 00:30:19 [notice] 1#1: OS: Linux 6.1.0-27-amd64
proxy | 2026/04/13 00:30:19 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
proxy | 2026/04/13 00:30:19 [notice] 1#1: start worker processes
proxy | 2026/04/13 00:30:19 [notice] 1#1: start worker process 28
proxy | 2026/04/13 00:30:19 [notice] 1#1: start worker process 29
```

Ahora debemos confirmar que después del levantamiento todos los pasos se completen estos pasos los podemos encontrar en el archivo ./app/Dockerfile

```
FROM python:3.10

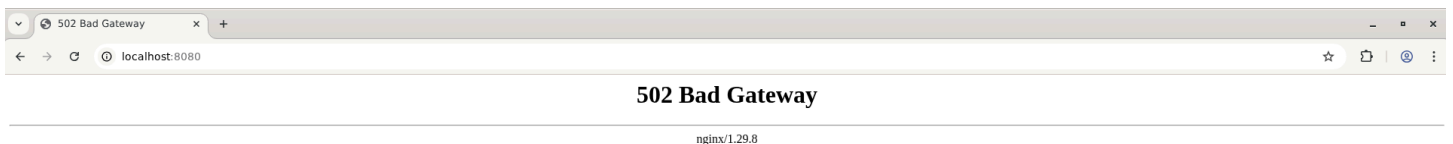
WORKDIR /app
COPY . .

RUN pip install -r requirements.txt

CMD ["python", "app.py"]
```

Tarea 2, 3 y 4: Identificación y Solución segundo error

Ya nuestro entorno en ejecución podemos proceder a probar via web que podamos acceder



Vemos que nos sale el error "502 Bad Gateway" en Nginx, tambien a comprobar los logs nos sale error.


```
proxy | 172.19.0.1 - - [13/Apr/2026:00:37:06 +0000] "GET / HTTP/1.1" 502 559 "-" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
proxy | 2026/04/13 00:37:06 [error] 29#29: *1 connect() failed (111: Connection refused) while connecting to upstream, client: 172.19.0.1, server: , request: "GET / HTTP/1.1", upstream: "http://172.19.0.2:8000/", host: "localhost:8080"
proxy | 172.19.0.1 - - [13/Apr/2026:00:37:06 +0000] "GET /favicon.ico HTTP/1.1" 502 559 "http://localhost:8080/" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
proxy | 2026/04/13 00:37:06 [error] 29#29: *1 connect() failed (111: Connection refused) while connecting to upstream, client: 172.19.0.1, server: , request: "GET /favicon.ico HTTP/1.1", upstream: "http://172.19.0.2:8000/favicon.ico", host: "localhost:8080", referer: "http://localhost:8080/"
```

Ahora debemos comprobar nuevamente el archivo "docker-compose.yml"

```
GNU nano 7.2 docker-compose.yml
version: '3.8'

services:
  ## web:
    backend:
      build: ./app
      container_name: app
      environment:
        - ENVIRONMENT=production
      expose:
        - "5000"
    nginx:
      image: nginx:latest
      container_name: proxy
      ports:
        - "8080:80"
      volumes:
        - ./nginx/default.conf:/etc/nginx/conf.d/default.conf
      depends_on:
        - backend
```

```
GNU nano 7.2 nginx/default.conf
server {
    listen 80;

    location / {
        proxy_pass http://backend:8000;
    }
}
```

Podemos evidenciar que la aplicacion escucha por el puerto 5000 y no por el puerto 8000 que esta en el archivo de ./nginx/default.conf

Como nginx actúa como proxy reverso utilizamos es el puerto 8080 para conectarnos desde un navegador remoto y comprobar su funcionamiento.

Con esto podemos deducir que esta mal configurado el puerto de escucha de la aplicacion ya que si vamos el archivo ./app./app.py vemos que en el código nos dice que la aplicacion escucha por un puerto diferente el 5000.

```
from flask import Flask
import os

app = Flask(__name__)

@app.route("/")
def hello():
    return f"Hello from {os.getenv('ENV', 'dev')}!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Para solucionar solo debemos entrar a ./nginx/default.conf ya cambiar del puerto 8000 al 5000

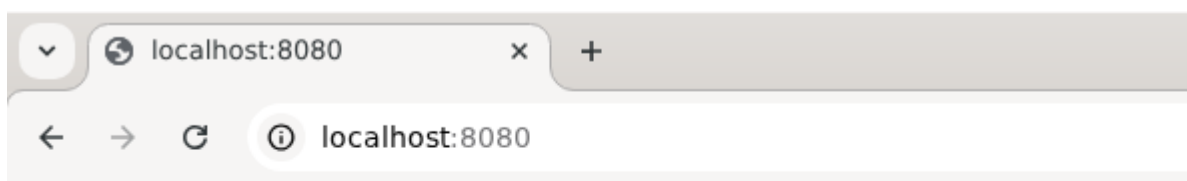
```
GNU nano 7.2 nginx/default.conf *
server {
    listen 80;

    location / {
##        proxy_pass http://backend:8000;
        proxy_pass http://backend:5000;
    }
}
```

guardamos y volvemos a levantar el entorno despues de detenerlo

```
deivi@debian:~/infra-test$ docker-compose up --build
Building backend
Sending build context to Docker daemon 7.168kB
Step 1/5 : FROM python:3.10
---> 738f70a7db01
Step 2/5 : WORKDIR /app
---> Using cache
---> 5d3f8adcffa5
Step 3/5 : COPY . .
---> 9380c96d9d2c
Step 4/5 : RUN pip install -r requirements.txt
---> Running in 7e586aa3ec90
Collecting flask
```

Volvemos a comprobar desde el navegador y evidenciamos que ya podemos conectarnos correctamente



Hello from dev!

Tarea 2, 3 y 4: Identificación y Solución tercer error

Podemos ver que el entorno está en funcionamiento pero se evidencia un error si comparamos el resultado desde el navegador nos muestra la palabra "Hello from dev!" ahora si voy nuevamente al archivo docker-compose.yml vemos que la variable de entorno está mal si la comparamos contra el archivo ./env

```
GNU nano 7.2                                     docker-compose.yml
version: '3.8'

services:
  ## web:
    backend:
      build: ./app
      container_name: app
      environment:
        - ENVIRONMENT=production
      expose:
        - "5000"

    nginx:
      image: nginx:latest
      container_name: proxy
      ports:
        - "8080:80"
      volumes:
        - ./nginx/default.conf:/etc/nginx/conf.d/default.conf
      depends_on:
        - backend
```

```
GNU nano 7.2                                     .env *
ENV=production
```

solo debemos corregir la variable por la correcta guardamos y volvemos a levantar el entorno

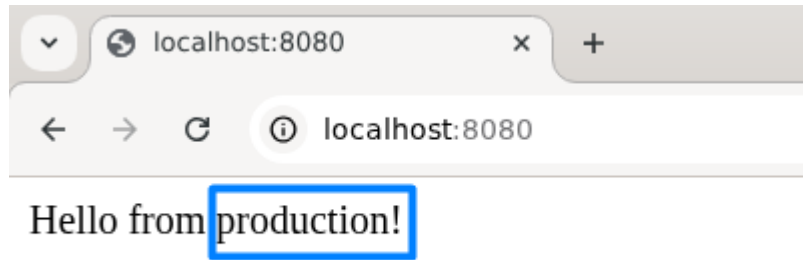
```
version: '3.8'

services:
  ## web:
    backend:
      build: ./app
      container_name: app
      environment:
        ## - ENVIRONMENT=production
        - ENV=production
      expose:
        - "5000"

    nginx:
      image: nginx:latest
      container_name: proxy
      ports:
        - "8080:80"
      volumes:
        - ./nginx/default.conf:/etc/nginx/conf.d/default.conf
      depends_on:
        - backend
```

```
deivi@debian:~/infra-test$ docker-compose up --build
Building backend
Sending build context to Docker daemon 7.168kB
Step 1/5 : FROM python:3.10
---> 738f70a7db01
Step 2/5 : WORKDIR /app
---> Using cache
---> 5d3f8adcffa5
Step 3/5 : COPY . .
---> Using cache
---> 9380c96d9d2c
Step 4/5 : RUN pip install -r requirements.txt
---> Using cache
---> b36043e0af15
Step 5/5 : CMD ["python", "app.py"]
---> Using cache
---> a3085fd8e7b2
Successfully built a3085fd8e7b2
Successfully tagged infra-test_backend:latest
Recreating app ... done
Recreating proxy ... done
Attaching to app, proxy
```

Si volvemos a comprobar confirmaremos que la variable de entorno ya esta en funcionamiento



Bonus

Mejorar Dockerfile

Se averiguo la mejor manera de optimizar este archivo donde están las instrucciones para construir la imagen que luego nos va a servir como plantilla para levantar el entorno

1. Optimización Profesional del Dockerfile:

- Uso de imágenes `slim` (reducción de peso).
- Implementación de **Multi-stage builds** (separar construcción de ejecución).
- Seguridad: Configuración de **usuario no-root** (appuser).
- Manejo de caché de capas (acelerar el build).

Para esto se realizo en otro entorno para diferenciar antes y después del bonus

```
deivi@debian:~$ cp -r infra-test infra-test-bonus
deivi@debian:~$ ls
Descargas  Escritorio  infra-test  infra-test.rar  my-docker-app  Público
Documentos  Imágenes   infra-test-bonus  Música          Plantillas     Vídeos
deivi@debian:~$ cd infra-test-bonus
deivi@debian:~/infra-test-bonus$ ls
app  docker-compose.yml  nginx  README.md
deivi@debian:~/infra-test-bonus$
```

Uso de imágenes `slim` (reducción de peso).

```
# 1. Imagen base ligera
FROM python:3.10-slim
```

```
# 2. Variables de entorno para Python
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# 3. Directorio de trabajo
WORKDIR /app

# 4. Seguridad: Crear un usuario no-root
RUN adduser --disabled-password --gecos "" appuser

# 5. Optimización de caché: Copiar solo dependencias primero
COPY requirements.txt .

# 6. Instalación limpia de librerías
RUN pip install --no-cache-dir -r requirements.txt

# 7. Copiar el resto del código y dar permisos
COPY . .
RUN chown -R appuser:appuser /app

# 8. Cambiar al usuario no-root
USER appuser

# 9. Comando de ejecución
CMD ["python", "app.py"]
```

```
GNU nano 7.2 app/Dockerfile *
##FROM python:3.10
##WORKDIR /app
##COPY . .
##RUN pip install -r requirements.txt
##CMD ["python", "app.py"]

# 1. Imagen base ligera
FROM python:3.10-slim

# 2. Variables de entorno para Python
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# 3. Directorio de trabajo
WORKDIR /app

# 4. Seguridad: Crear un usuario no-root
RUN adduser --disabled-password --gecos "" appuser

# 5. Optimización de caché: Copiar solo dependencias primero
COPY requirements.txt .

# 6. Instalación limpia de librerías
RUN pip install --no-cache-dir -r requirements.txt

# 7. Copiar el resto del código y dar permisos
COPY . .
RUN chown -R appuser:appuser /app

# 8. Cambiar al usuario no-root
USER appuser

# 9. Comando de ejecución
CMD ["python", "app.py"]
```

Explicación de cada paso:

1. Imagen `python:3.10-slim`

- **Por qué:** La imagen `python:3.10` original pesa unos **900MB**. La versión `slim` pesa unos **120MB**.
- **Beneficio:** Descargas más rápidas, menos espacio en disco y, sobre todo, **mayor seguridad** (tiene menos herramientas instaladas que un atacante podría usar).

```
deivi@debian:~/infra-test-bonus$ docker images
REPOSITORY          TAG          IMAGE ID      CREATED        SIZE
infra-test_backend  latest      a3085fd8e7b2  About an hour ago  1.11GB
<none>              <none>      ce43e8c78d6a  About an hour ago  1.11GB
<none>              <none>      94a86779d645  2 hours ago     1.11GB
```



```

deivi@debian:~/infra-test-bonus$ docker build -t test-app-pro ./app/
Sending build context to Docker daemon 6.144kB
Step 1/11 : FROM python:3.10-slim
3.10-slim: Pulling from library/python
5435b2dcdf5c: Already exists
935d60c8480a: Pull complete
bed04db721b4: Pull complete
ddf3b231f567: Pull complete
Digest: sha256:26fc00cf5fd6f57ded62e50e27cd3fec9843a3d0f31243c04debb3aba952d9c6
Status: Downloaded newer image for python:3.10-slim
--> 94717d9fb8a0
Step 2/11 : ENV PYTHONDONTWRITEBYTECODE=1
--> Running in bca9941025ce
Removing intermediate container bca9941025ce
--> 88233fb52153
Step 3/11 : ENV PYTHONUNBUFFERED=1
--> Running in 2323c7f1f6ec
Removing intermediate container 2323c7f1f6ec
--> 38612b7fe71e
Step 4/11 : WORKDIR /app
--> Running in 6afceb6de678
Removing intermediate container 6afceb6de678
--> 321564895f59
Step 5/11 : RUN adduser --disabled-password --gecos "" appuser
--> Running in 11156a0c87cb
usermod: no changes
Removing intermediate container 11156a0c87cb
--> f99f91a614f5
Step 6/11 : COPY requirements.txt .
--> 68fa255744a9
Step 7/11 : RUN pip install --no-cache-dir -r requirements.txt
--> Running in 9157fb8fc8d7
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl (103 kB)
    _____ 103.4/103.4 kB 7.2 MB/s eta 0:00:00
Collecting jinja2>=3.1.2
  Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
    _____ 134.9/134.9 kB 18.8 MB/s eta 0:00:00
Collecting werkzeug>=3.1.0

```

Despues de la construccion de la imagen nueva podemos observar que el tamaño se reduce considerablemente

```

deivi@debian:~/infra-test-bonus$ docker images
REPOSITORY          TAG          IMAGE ID          CREATED          SIZE
test-app-pro        latest      fb2fa343ea06     10 seconds ago  126MB
infra-test_backend  latest      a3085fd8e7b2     About an hour ago 1.11GB
<none>              <none>      ce43e8c78d6a     About an hour ago 1.11GB
<none>              <none>      94a86779d645     2 hours ago     1.11GB

```

2. Variables `PYTHONDONTWRITEBYTECODE` y `PYTHONUNBUFFERED`

- `PYTHONDONTWRITEBYTECODE=1`: Evita que Python escriba archivos `.pyc` en el disco del contenedor. Mantiene el contenedor limpio.

- **PYTHONUNBUFFERED=1**: Obliga a que los logs de Python se envíen directamente a la terminal de Docker sin pasar por un buffer.
- **Beneficio**: Si la app falla, verás el error en los logs **al instante**.

```
deivi@debian:~/infra-test-bonus$ docker exec app id
uid=1000(appuser) gid=1000(appuser) groups=1000(appuser),100(users)
deivi@debian:~/infra-test-bonus$ docker exec app env | grep PYTHON
PYTHON_VERSION=3.10.20
PYTHON_SHA256=de6517421601e39a9a3bc3e1bc4c7b2f239297423ee05e282598c83ec0647505
PYTHONDONTWRITEBYTECODE=1
PYTHONUNBUFFERED=1
```

1. **PYTHONUNBUFFERED=1** (El "Streaming" de Logs)

El Problema: Por defecto, Python es "tímido" con sus mensajes. En lugar de enviar cada línea de log a la consola en cuanto ocurre, las guarda en una memoria temporal (buffer). Espera a tener un grupo grande de mensajes para soltarlos todos de golpe.

En Docker: Si tu aplicación Flask tiene un error y se queda colgada, tú ejecutas `docker logs app` y no se ve nada. El error ocurrió, pero está atrapado en el buffer de Python y no ha salido a la terminal de Docker.

La Solución (`=1`): Desactiva el buffer. Obliga a Python a hacer un "streaming" en tiempo real.

2. **PYTHONDONTWRITEBYTECODE=1** (Limpieza de Disco)

El Problema: Cuando ejecutas un archivo `.py`, Python lo "traduce" a un lenguaje que la máquina entiende mejor y crea archivos llamados `.pyc` dentro de carpetas llamadas `__pycache__`. Esto sirve para que la *segunda vez* que abras la app, cargue un poco más rápido.

En Docker: Los contenedores son **efímeros**. Tú construyes la imagen una vez y la ejecutas mil veces. No queremos que el contenedor esté escribiendo archivos temporales innecesarios en su capa de escritura (que es lenta). Además, esos archivos `.pyc` pueden causar conflictos de permisos si cambias de usuario.

La Solución (`=1`): Le dice a Python: "No escribas nada extra en el disco, quédate solo con el código fuente que te di".

Resumen

"Uso `PYTHONUNBUFFERED` para asegurar que los logs de la aplicación se envíen en tiempo real a la salida estándar, permitiendo un monitoreo efectivo con `docker Logs`. Y uso `PYTHONDONTWRITEBYTECODE` para evitar la creación de archivos basura `.pyc` dentro del contenedor, manteniendo la imagen limpia y optimizada."

4 y 8. Usuario `appuser` (Seguridad Crítica)

- **Por qué:** Por defecto, Docker corre todo como `root`. Si un hacker logra explotar un fallo en tu app Flask, tendría control total sobre el contenedor y, potencialmente, sobre tu servidor Debian.
- **Beneficio:** Al usar `USER appuser`, la aplicación corre con permisos limitados. Es una de las mejores prácticas de seguridad más valoradas en entrevistas.

7. `chown -R appuser:appuser`

- **Por qué:** Como ahora usamos un usuario limitado, necesitamos asegurar que ese usuario sea el "dueño" de los archivos de la app para que pueda ejecutarlos.

Eliminación contenedores anteriores para comprobar el script actual

```
deivi@debian:~/infra-test-bonus$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS                    PORTS          NAMES
5c98e4c23143   nginx:latest   "/docker-entrypoint..." About an hour ago Exited (0) 50 minutes ago           proxy
01d007522063   infra-test_backend "python app.py"         About an hour ago Exited (137) 50 minutes ago         app
f94c0f18c2d6   wordpress:latest "docker-entrypoint.s..." 14 months ago Exited (0) 14 months ago           wordpress
1b2cb8aae8e8   mariadb:latest "docker-entrypoint.s..." 14 months ago Exited (0) 14 months ago           mariadb
a8cd96d77b89   my-python-app  "python3 hello.py"      14 months ago Exited (0) 14 months ago           crazy_murdock
fd2fec8c323b   hello-world    "/hello"                 14 months ago Exited (0) 14 months ago           mystifying_franklin
deivi@debian:~/infra-test-bonus$ docker rm app
app
deivi@debian:~/infra-test-bonus$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS                    PORTS          NAMES
5c98e4c23143   nginx:latest   "/docker-entrypoint..." About an hour ago Exited (0) 50 minutes ago           proxy
f94c0f18c2d6   wordpress:latest "docker-entrypoint.s..." 14 months ago Exited (0) 14 months ago           wordpress
1b2cb8aae8e8   mariadb:latest "docker-entrypoint.s..." 14 months ago Exited (0) 14 months ago           mariadb
a8cd96d77b89   my-python-app  "python3 hello.py"      14 months ago Exited (0) 14 months ago           crazy_murdock
fd2fec8c323b   hello-world    "/hello"                 14 months ago Exited (0) 14 months ago           mystifying_franklin
deivi@debian:~/infra-test-bonus$ docker rm proxy
proxy
deivi@debian:~/infra-test-bonus$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS                    PORTS          NAMES
f94c0f18c2d6   wordpress:latest "docker-entrypoint.s..." 14 months ago Exited (0) 14 months ago           wordpress
1b2cb8aae8e8   mariadb:latest "docker-entrypoint.s..." 14 months ago Exited (0) 14 months ago           mariadb
a8cd96d77b89   my-python-app  "python3 hello.py"      14 months ago Exited (0) 14 months ago           crazy_murdock
fd2fec8c323b   hello-world    "/hello"                 14 months ago Exited (0) 14 months ago           mystifying_franklin
deivi@debian:~/infra-test-bonus$
```

Comprobación del funcionamiento del script actual

```

deivi@debian:~/infra-test-bonus$ docker-compose up --build
Building backend
Sending build context to Docker daemon 6.144kB
Step 1/11 : FROM python:3.10-slim
--> 94717d9fb8a0
Step 2/11 : ENV PYTHONDONTWRITEBYTECODE=1
--> Using cache
--> 88233fb52153
Step 3/11 : ENV PYTHONUNBUFFERED=1
--> Using cache
--> 38612b7fe71e
Step 4/11 : WORKDIR /app
--> Using cache
--> 321564895f59
Step 5/11 : RUN adduser --disabled-password --gecos "" appuser
--> Using cache
--> f99f91a614f5
Step 6/11 : COPY requirements.txt .
--> Using cache
--> 68fa25744a9
Step 7/11 : RUN pip install --no-cache-dir -r requirements.txt
--> Using cache
--> 4bdd563a627d
Step 8/11 : COPY . .
--> Using cache
--> aaddcd5bf36
Step 9/11 : RUN chown -R appuser:appuser /app
--> Using cache
--> 515e391f8e58
Step 10/11 : USER appuser
--> Using cache
--> a054b17e81b9
Step 11/11 : CMD ["python", "app.py"]
--> Using cache
--> fb2fa343ea06
Successfully built fb2fa343ea06
Successfully tagged infra-test-bonus_backend:latest
Creating app ... done

```

Debemos confirmar que el proceso no lo corre el superusuario.

```

deivi@debian:~/infra-test-bonus$ docker exec app whoami
appuser

```

```

deivi@debian:~/infra-test-bonus$ docker exec app id
uid=1000(appuser) gid=1000(appuser) groups=1000(appuser),100(users)
deivi@debian:~/infra-test-bonus$

```

Validar Permisos de Archivos

```

deivi@debian:~/infra-test-bonus$ docker exec app ls -la /app
total 24
drwxr-xr-x 1 appuser appuser 4096 Apr 13 01:57 .
drwxr-xr-x 1 root    root    4096 Apr 13 02:09 ..
-rw-r--r-- 1 appuser appuser 1024 Apr 13 01:18 .app.py.swp
-rw-r--r-- 1 appuser appuser  753 Apr 13 01:48 Dockerfile
-rw-r--r-- 1 appuser appuser  206 Apr 13 01:18 app.py
-rw-r--r-- 1 appuser appuser   6  Apr 13 01:18 requirements.txt

```

5 y 6. El orden de `COPY` y `pip install`

- **Por qué:** Docker funciona por "capas" (layers). Si cambias una línea en tu `app.py`, Docker tiene que reconstruir desde donde hubo el cambio.
- **Truco:** Al copiar primero el `requirements.txt` e instalar, si mañana cambias el código pero no las librerías, Docker usará la "caché" y se saltará el paso de `pip install`.
- **Beneficio:** El build pasa de tardar 30 segundos a tardar 1 segundo.

Validar el Caché de Capas (Eficiencia de Build)

Esta es la prueba definitiva de que tu Dockerfile es rápido.

- **Acción:** Ejecuta `docker-compose build` nuevamente (sin cambiar nada).
- **Qué buscar:** En la terminal verás que casi todos los pasos dicen `CACHED`.
- **Prueba de fuego:** Cambia una letra en tu `app/app.py` y vuelve a ejecutar `docker-compose build`.
- **Qué buscar:** Verás que el paso de `pip install` se lo salta (**CACHED**) y solo ejecuta desde el `COPY . .` hacia abajo. Esto ahorra minutos de descarga en proyectos reales.

```

deivi@debian:~/infra-test-bonus$ docker-compose up --build
Building backend
Sending build context to Docker daemon 6.144kB
Step 1/11 : FROM python:3.10-slim
--> 94717d9fb8a0
Step 2/11 : ENV PYTHONDONTWRITEBYTECODE=1
--> Using cache
--> 88233fb52153
Step 3/11 : ENV PYTHONUNBUFFERED=1
--> Using cache
--> 38612b7fe71e
Step 4/11 : WORKDIR /app

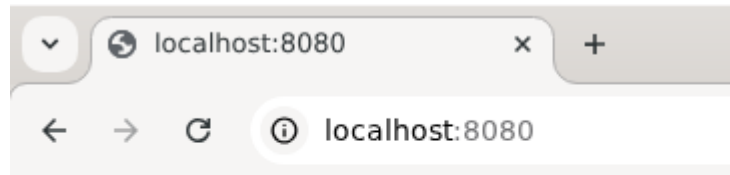
```

```
---> Using cache
---> 321564895f59
Step 5/11 : RUN adduser --disabled-password --gecos "" appuser
---> Using cache
---> f99f91a614f5
Step 6/11 : COPY requirements.txt .
---> Using cache
---> 68fa255744a9
Step 7/11 : RUN pip install --no-cache-dir -r requirements.txt
---> Using cache
---> 4bdd563a627d
Step 8/11 : COPY . .
---> Using cache
---> aaddcd5bf36
Step 9/11 : RUN chown -R appuser:appuser /app
---> Using cache
---> 515e391f8e58
Step 10/11 : USER appuser
---> Using cache
---> a054b17e81b9
Step 11/11 : CMD ["python", "app.py"]
---> Using cache
---> fb2fa343ea06
Successfully built fb2fa343ea06
Successfully tagged infra-test-bonus_backend:latest
Creating app ... done
Creating proxy ... done
Attaching to app, proxy
proxy    | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
proxy    | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
proxy    | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
proxy    | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
proxy    | 10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf differs from the packaged version
proxy    | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
proxy    | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
proxy    | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
proxy    | /docker-entrypoint.sh: Configuration complete; ready for start up
proxy    | 2026/04/13 02:10:01 [notice] 1#1: using the "epoll" event method
proxy    | 2026/04/13 02:10:01 [notice] 1#1: nginx/1.29.8
proxy    | 2026/04/13 02:10:01 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
proxy    | 2026/04/13 02:10:01 [notice] 1#1: OS: Linux 6.1.0-27-amd64
proxy    | 2026/04/13 02:10:01 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
proxy    | 2026/04/13 02:10:01 [notice] 1#1: start worker processes
proxy    | 2026/04/13 02:10:01 [notice] 1#1: start worker process 28
proxy    | 2026/04/13 02:10:01 [notice] 1#1: start worker process 29
app      | * Serving Flask app 'app'
app      | * Debug mode: off
app      | WARNING: This is a development server. Do not use it in a production deployment. Use a production
WSGI server instead.
```



```
app | * Running on all addresses (0.0.0.0)
app | * Running on http://127.0.0.1:5000
app | * Running on http://172.20.0.2:5000
app | Press CTRL+C to quit
```

Todo esta en funcionamiento



Hello from production!

```
deivi@debian:~/infra-test-bonus$ curl -f http://localhost:8080
Hello from production!deivi@debian:~/infra-test-bonus$
```

```
proxy | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
proxy | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
proxy | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
proxy | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
proxy | 10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf differs from the packaged version
proxy | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
proxy | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
proxy | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
proxy | /docker-entrypoint.sh: Configuration complete; ready for start up
proxy | 2026/04/13 02:10:01 [notice] 1#1: using the "epoll" event method
proxy | 2026/04/13 02:10:01 [notice] 1#1: nginx/1.29.8
proxy | 2026/04/13 02:10:01 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
proxy | 2026/04/13 02:10:01 [notice] 1#1: OS: Linux 6.1.0-27-amd64
proxy | 2026/04/13 02:10:01 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
proxy | 2026/04/13 02:10:01 [notice] 1#1: start worker processes
proxy | 2026/04/13 02:10:01 [notice] 1#1: start worker process 28
proxy | 2026/04/13 02:10:01 [notice] 1#1: start worker process 29
app | * Serving Flask app 'app'
app | * Debug mode: off
app | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
app | * Running on all addresses (0.0.0.0)
app | * Running on http://127.0.0.1:5000
app | * Running on http://172.20.0.2:5000
app | Press CTRL+C to quit
proxy | 172.20.0.1 - - [13/Apr/2026:02:19:59 +0000] "GET / HTTP/1.1" 200 22 "-" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
app | 172.20.0.3 - - [13/Apr/2026 02:19:59] "GET / HTTP/1.1" 200 -
app | 172.20.0.3 - - [13/Apr/2026 02:20:00] "GET / HTTP/1.1" 200 -
proxy | 172.20.0.1 - - [13/Apr/2026:02:20:00 +0000] "GET / HTTP/1.1" 200 22 "-" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
proxy | 172.20.0.1 - - [13/Apr/2026:02:20:00 +0000] "GET / HTTP/1.1" 200 22 "-" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
app | 172.20.0.3 - - [13/Apr/2026 02:20:00] "GET / HTTP/1.1" 200 -
app | 172.20.0.3 - - [13/Apr/2026 02:20:00] "GET / HTTP/1.1" 200 -
proxy | 172.20.0.1 - - [13/Apr/2026:02:20:00 +0000] "GET / HTTP/1.1" 200 22 "-" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"
```

Agregar Healthcheck

¿Qué es y para qué sirve?

En el mundo de los servidores, un proceso puede estar "corriendo" (UP) pero estar "congelado" (no responde peticiones).

- **Sin Healthcheck:** Docker ve que el proceso Python existe y dice "todo está bien", aunque no puedan entrar a la web.
- **Con Healthcheck:** Docker intenta "tocar la puerta" de la app cada cierto tiempo. Si la app no responde, Docker marca el contenedor como **unhealthy** (no sano) y puede reiniciarlo automáticamente o dejar de enviarle tráfico.

```
deivi@debian:~/infra-test-bonus$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
b98e6f5f3df9   nginx:latest   "/docker-entrypoint..." 20 minutes ago Up 20 minutes 0.0.0.0:8080->80/tcp, :::8080->80/tcp
824df83f8900   infra-test-bonus_backend "python app.py"         20 minutes ago Up 20 minutes 5000/tcp
```

Paso 1: Modificar el Dockerfile (Instalar la herramienta)

Para que Docker pueda probar la app, necesita una herramienta para hacer peticiones internas. Usaremos **curl**.

En tu **app/Dockerfile**, añade **curl** en la sección donde instalas paquetes (si usas la versión optimizada que te di, quedaría así):

El código se debe mover a la parte superior del archivo Dockerfile ya que si se hace a lo último el script falla por permisos o sea por el #8 cambias a usuario no-root y se debe hacer luego del from ya que es lo primero que se debe ejecutar y también para que quede en cache


```
GNU nano 7.2 app/Dockerfile
##FROM python:3.10
##WORKDIR /app
##COPY . .
##RUN pip install -r requirements.txt
##CMD ["python", "app.py"]

# 1. Imagen base ligera
FROM python:3.10-slim

# Bonus
# Healthcheck Para que Docker pueda probar el funcionamiento de la app cada cierto tiempo.
RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*

# 2. Variables de entorno para Python
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# 3. Directorio de trabajo
WORKDIR /app

# 4. Seguridad: Crear un usuario no-root
RUN adduser --disabled-password --gecos "" appuser

# 5. Optimización de caché: Copiar solo dependencias primero
COPY requirements.txt .

# 6. Instalación limpia de librerías
RUN pip install --no-cache-dir -r requirements.txt

# 7. Copiar el resto del código y dar permisos
COPY . .
RUN chown -R appuser:appuser /app

# 8. Cambiar al usuario no-root
USER appuser
```

Paso 2: Configurar el Healthcheck en `docker-compose.yml`

Añadiremos la lógica de monitoreo directamente en el orquestador.

Modifica tu `docker-compose.yml` en el servicio `backend`:

```
GNU nano 7.2                                     docker-compose.yml *
```

```
version: '3.8'

services:
  ## web:
    backend:
      build: ./app
      container_name: app
      environment:
      ##
        - ENVIRONMENT=production
        - ENV=production
      expose:
        - "5000"
  ## Healthcheck
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:5000/"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 10s

  nginx:
    image: nginx:latest
    container_name: proxy
    ports:
      - "8080:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf
    depends_on:
      - backend
```

Explicación de las banderas:

- **test**: El comando que Docker ejecuta dentro del contenedor. Si el **curl** falla, el test falla.
- **interval**: Realiza la prueba cada 30 segundos.
- **timeout**: Si la app tarda más de 10 segundos en responder, se considera fallo.
- **retries**: Si falla 3 veces seguidas, el contenedor pasa a estado **unhealthy**.
- **start_period**: Le da a la app 10 segundos para arrancar antes de empezar a evaluarla (evita marcarla como fallida mientras carga Python).

1. Aplica los cambios:

```
deivi@debian:~/infra-test-bonus$ docker-compose up -d --build
Building backend
Sending build context to Docker daemon 6.144kB
Step 1/12 : FROM python:3.10-slim
---> 94717d9fb8a0
```

Step 2/12 : RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*

---> Running in 40fb1b255ba2

Hit:1 http://deb.debian.org/debian trixie InRelease

Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]

Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]

Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9671 kB]

Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages [5412 B]

Get:6 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [122 kB]

Fetchd 9889 kB in 1s (8203 kB/s)

Reading package lists...

Reading package lists...

Building dependency tree...

Reading state information...

The following additional packages will be installed:

bash-completion krb5-locales libbrotli1 libcom-err2 libcurl4t64

libgnutls30t64 libgssapi-krb5-2 libidn2-0 libk5crypto3 libkeyutils1

libkrb5-3 libkrb5support0 libldap-common libldap2 libnghttp2-14 libnghttp3-9

libp11-kit0 libpsl5t64 librtmp1 libsasl2-2 libsasl2-modules

libsasl2-modules-db libssh2-1t64 libtasn1-6 libunistring5 publicsuffix

Suggested packages:

gnutls-bin krb5-doc krb5-user libsasl2-modules-gssapi-mit

| libsasl2-modules-gssapi-heimdal libsasl2-modules-ldap libsasl2-modules-otp

libsasl2-modules-sql

The following NEW packages will be installed:

bash-completion curl krb5-locales libbrotli1 libcom-err2 libcurl4t64

libgnutls30t64 libgssapi-krb5-2 libidn2-0 libk5crypto3 libkeyutils1

libkrb5-3 libkrb5support0 libldap-common libldap2 libnghttp2-14 libnghttp3-9

libp11-kit0 libpsl5t64 librtmp1 libsasl2-2 libsasl2-modules

libsasl2-modules-db libssh2-1t64 libtasn1-6 libunistring5 publicsuffix

0 upgraded, 27 newly installed, 0 to remove and 3 not upgraded.

Need to get 5703 kB of archives.

After this operation, 18.3 MB of additional disk space will be used.

Get:1 http://deb.debian.org/debian trixie/main amd64 bash-completion all 1:2.16.0-7 [319 kB]

Get:2 http://deb.debian.org/debian trixie/main amd64 krb5-locales all 1.21.3-5 [101 kB]

Get:3 http://deb.debian.org/debian trixie/main amd64 libbrotli1 amd64 1.1.0-2+b7 [307 kB]

Get:4 http://deb.debian.org/debian trixie/main amd64 libkrb5support0 amd64 1.21.3-5 [33.0 kB]

Get:5 http://deb.debian.org/debian trixie/main amd64 libcom-err2 amd64 1.47.2-3+b10 [25.0 kB]

Get:6 http://deb.debian.org/debian trixie/main amd64 libk5crypto3 amd64 1.21.3-5 [81.5 kB]

Get:7 http://deb.debian.org/debian trixie/main amd64 libkeyutils1 amd64 1.6.3-6 [9456 B]

Get:8 http://deb.debian.org/debian trixie/main amd64 libkrb5-3 amd64 1.21.3-5 [326 kB]

Get:9 http://deb.debian.org/debian trixie/main amd64 libgssapi-krb5-2 amd64 1.21.3-5 [138 kB]

Get:10 http://deb.debian.org/debian trixie/main amd64 libunistring5 amd64 1.3-2 [477 kB]

Get:11 http://deb.debian.org/debian trixie/main amd64 libidn2-0 amd64 2.3.8-2 [109 kB]

Get:12 http://deb.debian.org/debian trixie/main amd64 libsasl2-modules-db amd64 2.1.28+dfsg1-9 [19.8 kB]

Get:13 http://deb.debian.org/debian trixie/main amd64 libsasl2-2 amd64 2.1.28+dfsg1-9 [57.5 kB]

Get:14 http://deb.debian.org/debian trixie/main amd64 libldap2 amd64 2.6.10+dfsg-1 [194 kB]

Get:15 http://deb.debian.org/debian trixie/main amd64 libnghttp2-14 amd64 1.64.0-1.1 [76.0 kB]

Get:16 http://deb.debian.org/debian trixie/main amd64 libnghttp3-9 amd64 1.8.0-1 [67.7 kB]

Get:17 http://deb.debian.org/debian trixie/main amd64 libpsl5t64 amd64 0.21.2-1.1+b1 [57.2 kB]

Get:18 <http://deb.debian.org/debian trixie/main amd64 libp11-kit0 amd64 0.25.5-3> [425 kB]
Get:19 <http://deb.debian.org/debian trixie/main amd64 libtasn1-6 amd64 4.20.0-2> [49.9 kB]
Get:20 <http://deb.debian.org/debian trixie/main amd64 libgnutls30t64 amd64 3.8.9-3+deb13u2> [1468 kB]
Get:21 <http://deb.debian.org/debian trixie/main amd64 librtmp1 amd64 2.4+20151223.gitfa8646d.1-2+b5> [58.8 kB]
Get:22 <http://deb.debian.org/debian trixie/main amd64 libssh2-1t64 amd64 1.11.1-1> [245 kB]
Get:23 <http://deb.debian.org/debian trixie/main amd64 libcurl4t64 amd64 8.14.1-2+deb13u2> [391 kB]
Get:24 <http://deb.debian.org/debian trixie/main amd64 curl amd64 8.14.1-2+deb13u2> [270 kB]
Get:25 <http://deb.debian.org/debian trixie/main amd64 libldap-common all 2.6.10+dfsg-1> [35.1 kB]
Get:26 <http://deb.debian.org/debian trixie/main amd64 libsasl2-modules amd64 2.1.28+dfsg1-9> [66.7 kB]
Get:27 <http://deb.debian.org/debian trixie/main amd64 publicsuffix all 20250328.1952-0.1> [296 kB]
debconf: unable to initialize frontend: Dialog
debconf: (TERM is not set, so the dialog frontend is not usable.)
debconf: falling back to frontend: Readline
debconf: unable to initialize frontend: Readline
debconf: (Can't locate Term/ReadLine.pm in @INC (you may need to install the Term::ReadLine module) (@INC entries checked: /etc/perl /usr/local/lib/x86_64-linux-gnu/perl/5.40.1 /usr/local/share/perl/5.40.1 /usr/lib/x86_64-linux-gnu/perl5/5.40 /usr/share/perl5 /usr/lib/x86_64-linux-gnu/perl-base /usr/lib/x86_64-linux-gnu/perl/5.40 /usr/share/perl/5.40 /usr/local/lib/site_perl) at /usr/share/perl5/Debconf/FrontEnd/Readline.pm line 8, <STDIN> line 27.)
debconf: falling back to frontend: Teletype
debconf: unable to initialize frontend: Teletype
debconf: (This frontend requires a controlling tty.)
debconf: falling back to frontend: Noninteractive
Fetched 5703 kB in 0s (23.1 MB/s)
Selecting previously unselected package bash-completion.
(Reading database ... 5645 files and directories currently installed.)
Preparing to unpack .../00-bash-completion_1%3a2.16.0-7_all.deb ...
Unpacking bash-completion (1:2.16.0-7) ...
Selecting previously unselected package krb5-locales.
Preparing to unpack .../01-krb5-locales_1.21.3-5_all.deb ...
Unpacking krb5-locales (1.21.3-5) ...
Selecting previously unselected package libbrotli1:amd64.
Preparing to unpack .../02-libbrotli1_1.1.0-2+b7_amd64.deb ...
Unpacking libbrotli1:amd64 (1.1.0-2+b7) ...
Selecting previously unselected package libkrb5support0:amd64.
Preparing to unpack .../03-libkrb5support0_1.21.3-5_amd64.deb ...
Unpacking libkrb5support0:amd64 (1.21.3-5) ...
Selecting previously unselected package libcom-err2:amd64.
Preparing to unpack .../04-libcom-err2_1.47.2-3+b10_amd64.deb ...
Unpacking libcom-err2:amd64 (1.47.2-3+b10) ...
Selecting previously unselected package libk5crypto3:amd64.
Preparing to unpack .../05-libk5crypto3_1.21.3-5_amd64.deb ...
Unpacking libk5crypto3:amd64 (1.21.3-5) ...
Selecting previously unselected package libkeyutils1:amd64.
Preparing to unpack .../06-libkeyutils1_1.6.3-6_amd64.deb ...
Unpacking libkeyutils1:amd64 (1.6.3-6) ...
Selecting previously unselected package libkrb5-3:amd64.
Preparing to unpack .../07-libkrb5-3_1.21.3-5_amd64.deb ...
Unpacking libkrb5-3:amd64 (1.21.3-5) ...

Selecting previously unselected package libgssapi-krb5-2:amd64.
Preparing to unpack .../08-libgssapi-krb5-2_1.21.3-5_amd64.deb ...
Unpacking libgssapi-krb5-2:amd64 (1.21.3-5) ...
Selecting previously unselected package libunistring5:amd64.
Preparing to unpack .../09-libunistring5_1.3-2_amd64.deb ...
Unpacking libunistring5:amd64 (1.3-2) ...
Selecting previously unselected package libidn2-0:amd64.
Preparing to unpack .../10-libidn2-0_2.3.8-2_amd64.deb ...
Unpacking libidn2-0:amd64 (2.3.8-2) ...
Selecting previously unselected package libsasl2-modules-db:amd64.
Preparing to unpack .../11-libsasl2-modules-db_2.1.28+dfsg1-9_amd64.deb ...
Unpacking libsasl2-modules-db:amd64 (2.1.28+dfsg1-9) ...
Selecting previously unselected package libsasl2-2:amd64.
Preparing to unpack .../12-libsasl2-2_2.1.28+dfsg1-9_amd64.deb ...
Unpacking libsasl2-2:amd64 (2.1.28+dfsg1-9) ...
Selecting previously unselected package libldap2:amd64.
Preparing to unpack .../13-libldap2_2.6.10+dfsg-1_amd64.deb ...
Unpacking libldap2:amd64 (2.6.10+dfsg-1) ...
Selecting previously unselected package libnghttp2-14:amd64.
Preparing to unpack .../14-libnghttp2-14_1.64.0-1.1_amd64.deb ...
Unpacking libnghttp2-14:amd64 (1.64.0-1.1) ...
Selecting previously unselected package libnghttp3-9:amd64.
Preparing to unpack .../15-libnghttp3-9_1.8.0-1_amd64.deb ...
Unpacking libnghttp3-9:amd64 (1.8.0-1) ...
Selecting previously unselected package libpsl5t64:amd64.
Preparing to unpack .../16-libpsl5t64_0.21.2-1.1+b1_amd64.deb ...
Unpacking libpsl5t64:amd64 (0.21.2-1.1+b1) ...
Selecting previously unselected package libp11-kit0:amd64.
Preparing to unpack .../17-libp11-kit0_0.25.5-3_amd64.deb ...
Unpacking libp11-kit0:amd64 (0.25.5-3) ...
Selecting previously unselected package libtasn1-6:amd64.
Preparing to unpack .../18-libtasn1-6_4.20.0-2_amd64.deb ...
Unpacking libtasn1-6:amd64 (4.20.0-2) ...
Selecting previously unselected package libgnutls30t64:amd64.
Preparing to unpack .../19-libgnutls30t64_3.8.9-3+deb13u2_amd64.deb ...
Unpacking libgnutls30t64:amd64 (3.8.9-3+deb13u2) ...
Selecting previously unselected package librtmp1:amd64.
Preparing to unpack .../20-librtmp1_2.4+20151223.gitfa8646d.1-2+b5_amd64.deb ...
Unpacking librtmp1:amd64 (2.4+20151223.gitfa8646d.1-2+b5) ...
Selecting previously unselected package libssh2-1t64:amd64.
Preparing to unpack .../21-libssh2-1t64_1.11.1-1_amd64.deb ...
Unpacking libssh2-1t64:amd64 (1.11.1-1) ...
Selecting previously unselected package libcurl4t64:amd64.
Preparing to unpack .../22-libcurl4t64_8.14.1-2+deb13u2_amd64.deb ...
Unpacking libcurl4t64:amd64 (8.14.1-2+deb13u2) ...
Selecting previously unselected package curl.
Preparing to unpack .../23-curl_8.14.1-2+deb13u2_amd64.deb ...
Unpacking curl (8.14.1-2+deb13u2) ...
Selecting previously unselected package libldap-common.

```
Preparing to unpack .../24-libldap-common_2.6.10+dfsg-1_all.deb ...
Unpacking libldap-common (2.6.10+dfsg-1) ...
Selecting previously unselected package libsasl2-modules:amd64.
Preparing to unpack .../25-libsasl2-modules_2.1.28+dfsg1-9_amd64.deb ...
Unpacking libsasl2-modules:amd64 (2.1.28+dfsg1-9) ...
Selecting previously unselected package publicsuffix.
Preparing to unpack .../26-publicsuffix_20250328.1952-0.1_all.deb ...
Unpacking publicsuffix (20250328.1952-0.1) ...
Setting up libkeyutils1:amd64 (1.6.3-6) ...
Setting up libbrotli1:amd64 (1.1.0-2+b7) ...
Setting up libsasl2-modules:amd64 (2.1.28+dfsg1-9) ...
Setting up libnghttp2-14:amd64 (1.64.0-1.1) ...
Setting up krb5-locales (1.21.3-5) ...
Setting up libcom-err2:amd64 (1.47.2-3+b10) ...
Setting up libldap-common (2.6.10+dfsg-1) ...
Setting up libkrb5support0:amd64 (1.21.3-5) ...
Setting up libsasl2-modules-db:amd64 (2.1.28+dfsg1-9) ...
Setting up bash-completion (1:2.16.0-7) ...
Setting up libp11-kit0:amd64 (0.25.5-3) ...
Setting up libunistring5:amd64 (1.3-2) ...
Setting up libk5crypto3:amd64 (1.21.3-5) ...
Setting up libsasl2-2:amd64 (2.1.28+dfsg1-9) ...
Setting up libnghttp3-9:amd64 (1.8.0-1) ...
Setting up libtasn1-6:amd64 (4.20.0-2) ...
Setting up libkrb5-3:amd64 (1.21.3-5) ...
Setting up libssh2-1t64:amd64 (1.11.1-1) ...
Setting up publicsuffix (20250328.1952-0.1) ...
Setting up libldap2:amd64 (2.6.10+dfsg-1) ...
Setting up libidn2-0:amd64 (2.3.8-2) ...
Setting up libgssapi-krb5-2:amd64 (1.21.3-5) ...
Setting up libgnutls30t64:amd64 (3.8.9-3+deb13u2) ...
Setting up libpsl5t64:amd64 (0.21.2-1.1+b1) ...
Setting up librtmp1:amd64 (2.4+20151223.gitfa8646d.1-2+b5) ...
Setting up libcurl4t64:amd64 (8.14.1-2+deb13u2) ...
Setting up curl (8.14.1-2+deb13u2) ...
Processing triggers for libc-bin (2.41-12+deb13u2) ...
Removing intermediate container 40fb1b255ba2
---> 6438b920efeb
Step 3/12 : ENV PYTHONDONTWRITEBYTECODE=1
---> Running in 121d8f038f96
Removing intermediate container 121d8f038f96
---> 26c237c74473
Step 4/12 : ENV PYTHONUNBUFFERED=1
---> Running in 44ba20f7a390
Removing intermediate container 44ba20f7a390
---> de938a75f052
Step 5/12 : WORKDIR /app
---> Running in b7248dc33b6d
Removing intermediate container b7248dc33b6d
```

```
---> 47cbf944879b
Step 6/12 : RUN adduser --disabled-password --gecos "" appuser
---> Running in a4e677e4838c
usermod: no changes
Removing intermediate container a4e677e4838c
---> 357f3cec824a
Step 7/12 : COPY requirements.txt .
---> 6be9a7fdbb32
Step 8/12 : RUN pip install --no-cache-dir -r requirements.txt
---> Running in b620778a8fb3
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl (103 kB)


---


103.4/103.4 kB 23.2 MB/s eta 0:00:00
Collecting werkzeug>=3.1.0
  Downloading werkzeug-3.1.8-py3-none-any.whl (226 kB)


---


226.5/226.5 kB 48.3 MB/s eta 0:00:00
Collecting blinker>=1.9.0
  Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Collecting markupsafe>=2.1.1
  Downloading markupsafe-3.0.3-cp310-cp310-
manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (20 kB)
Collecting click>=8.1.3
  Downloading click-8.3.2-py3-none-any.whl (108 kB)


---


108.4/108.4
kB 193.1 MB/s eta 0:00:00
Collecting jinja2>=3.1.2
  Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)


---


134.9/134.9 kB 82.2 MB/s eta 0:00:00
Collecting itsdangerous>=2.2.0
  Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, flask
Successfully installed blinker-1.9.0 click-8.3.2 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3
werkzeug-3.1.8
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system
package manager. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv

[notice] A new release of pip is available: 23.0.1 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
Removing intermediate container b620778a8fb3
---> f223dcf4d906
Step 9/12 : COPY . .
---> c7299a1a637d
Step 10/12 : RUN chown -R appuser:appuser /app
---> Running in 94be6bc24058
Removing intermediate container 94be6bc24058
```

```

---> 9d0a77778763
Step 11/12 : USER appuser
---> Running in 0d5fb12ced0a
Removing intermediate container 0d5fb12ced0a
---> af473b87b0de
Step 12/12 : CMD ["python", "app.py"]
---> Running in 4b2cd838c8a1
Removing intermediate container 4b2cd838c8a1
---> c1ec25a37d6d
Successfully built c1ec25a37d6d
Successfully tagged infra-test-bonus_backend:latest
Recreating app ... done
Recreating proxy ... done

```

Validación (Cómo saber si funcionó):

Ejecuta el comando:

```
docker ps
```

Qué buscar:

En la columna **STATUS**, verás algo como:

- **Up 10 seconds (health: starting)** -> Está esperando el **start_period**.
- **Up 1 minute (healthy)** -> ¡Éxito! La app respondió correctamente al test.
- **Up 1 minute (unhealthy)** -> Algo falló (revisa puertos o si Flask está caído).

```

deivi@debian:~/infra-test-bonus$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS              PORTS
c010beea1fb9   nginx:latest   "/docker-entrypoint..." 2 minutes ago  Up 2 minutes       0.0.0.0:8080->80/tcp, :::8080->80/tcp
p_proxy        6304d92cb69e   infra-test-bonus_backend "python app.py"       2 minutes ago  Up 2 minutes (healthy)  5000/tcp
app

```

Prueba (Simular fallo):

Si detienes el proceso de Flask por dentro o cambias el puerto, verás que Docker lo marca como **unhealthy** sin que el contenedor se detenga.

```

## Healthcheck
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:9999/"]
  interval: 30s
  timeout: 10s
  retries: 3
  start_period: 10s

```



```

deivi@debian:~/infra-test-bonus$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
NAMES
c010beea1fb9   nginx:latest   "/docker-entrypoint..." 4 minutes ago  Up 4 minutes   0.0.0.0:8080->80/tcp, :::8080->80/tcp
p_proxy
6304d92cb69e   infra-test-bonus_backend "python app.py"         4 minutes ago  Up 4 minutes (healthy)  5000/tcp
app
deivi@debian:~/infra-test-bonus$ docker st
stack start stats stop
deivi@debian:~/infra-test-bonus$ docker stop app proxy
app
proxy
deivi@debian:~/infra-test-bonus$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
deivi@debian:~/infra-test-bonus$ docker-compose up -d --build
Building backend
Sending build context to Docker daemon  6.144kB
Step 1/12 : FROM python:3.10-slim
----> 94717d9fb8a0
Step 2/12 : RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*
----> Using cache
----> 6438b920efeb

```

```

deivi@debian:~/infra-test-bonus$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
NAMES
8de97d8c1d07   nginx:latest   "/docker-entrypoint..." 5 seconds ago  Up 4 seconds   0.0.0.0:8080->80/tcp, :::80
2157134f6a3b   infra-test-bonus_backend "python app.py"         6 seconds ago  Up 4 seconds (health: starting)  5000/tcp
app

```

```

deivi@debian:~/infra-test-bonus$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
NAMES
8de97d8c1d07   nginx:latest   "/docker-entrypoint..." About a minute ago  Up About a minute   0.0.0.0:8080->80/tcp, :::80
2157134f6a3b   infra-test-bonus_backend "python app.py"         About a minute ago  Up About a minute (unhealthy)  5000/tcp
app
deivi@debian:~/infra-test-bonus$

```

Logs Claros

La prioridad es que los logs sean **seguros** (que no llenen el disco) y **accesibles** (que no se pierdan si el contenedor muere). Manejaremos todo desde la capa de Docker.

Paso 1: Configurar la Rotación de Logs (Seguridad de Disco)

Por defecto, Docker guarda logs infinitamente hasta agotar el espacio en disco de tu Debian. En producción, esto es un error fatal.

Añadiremos una política de rotación en el `docker-compose.yml` para ambos servicios (`backend` y `nginx`):

```
GNU nano 7.2                                     docker-compose.yml *
```

```
  container_name: app
  environment:
##    - ENVIRONMENT=production
    - ENV=production
  expose:
    - "5000"
## Healthcheck
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:5000/"]
    interval: 30s
    timeout: 10s
    retries: 3
    start_period: 10s

## Logs Claros
  logging:
    driver: "json-file"
    options:
      max-size: "10m" # El archivo de log no pasará de 10MB
      max-file: "3"   # Solo guardará los últimos 3 archivos

  nginx:
    image: nginx:latest
    container_name: proxy
    ports:
      - "8080:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf
    depends_on:
      - backend

## Logs Claros
  logging:
    driver: "json-file"
    options:
      max-size: "10m" # El archivo de log no pasará de 10MB
      max-file: "3"   # Solo guardará los últimos 3 archivos
```

Paso 2: Persistencia de Logs (Mapeo al Host)

Para que puedas auditar los logs desde tu Debian sin usar comandos de Docker, vamos a mapear los logs de Nginx a una carpeta real en tu máquina virtual.

1. Crea la carpeta en Debian para los logs:

```
deivi@debian:~/infra-test-bonus$ mkdir -p logs/nginx
deivi@debian:~/infra-test-bonus$ ls logs/
nginx
```

2. Modificar la sección de `volumes` del servicio `nginx` en el `docker-compose.yml`:

```
- ./logs/nginx/var/log/nginx # Mapea los logs internos al host
```

```
nginx:
  image: nginx:latest
  container_name: proxy
  ports:
    - "8080:80"
  volumes:
    - ./nginx/default.conf:/etc/nginx/conf.d/default.conf
    - ./logs/nginx:/var/log/nginx
  depends_on:
    - backend
## Logs Claros
logging:
  driver: "json-file"
  options:
    max-size: "10m" # El archivo de log no pasará de 10MB
    max-file: "3"   # Solo guardará los últimos 3 archivos
```

Tarea para tu VM Debian 12:

1. Aplica los cambios en el `docker-compose.yml` (Rotación y Volúmenes).
2. Reinicia el entorno:

```
deivi@debian:~/infra-test-bonus$ docker-compose up -d --build
Building backend
Sending build context to Docker daemon 4.608kB
Step 1/12 : FROM python:3.10-slim
--> 94717d9fb8a0
Step 2/12 : RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*
--> Using cache
--> 6438b920efeb
Step 3/12 : ENV PYTHONDONTWRITEBYTECODE=1
--> Using cache
--> 26c237c74473
Step 4/12 : ENV PYTHONUNBUFFERED=1
--> Using cache
--> de938a75f052
Step 5/12 : WORKDIR /app
--> Using cache
--> 47cbf944879b
Step 6/12 : RUN adduser --disabled-password --gecos "" appuser
--> Using cache
--> 357f3cec824a
Step 7/12 : COPY requirements.txt .
--> Using cache
--> 6be9a7fdbb32
Step 8/12 : RUN pip install --no-cache-dir -r requirements.txt
--> Using cache
--> f223dcf4d906
Step 9/12 : COPY . .
--> bfe8e889e5b3
Step 10/12 : RUN chown -R appuser:appuser /app
--> Running in c7b7a02ab91a
Removing intermediate container c7b7a02ab91a
--> d09610bb077d
Step 11/12 : USER appuser
--> Running in aa91290996cc
Removing intermediate container aa91290996cc
--> c4cf0c7c1158
Step 12/12 : CMD ["python", "app.py"]
--> Running in ec5d84da0f8c
Removing intermediate container ec5d84da0f8c
--> c8af868dd611
Successfully built c8af868dd611
Successfully tagged infra-test-bonus_backend:latest
Recreating app ... done
Recreating proxy ... done
```

Validación (Cómo comprobar los logs):

1. Ver logs en tiempo real (Docker):

Ejecutamos este comando y luego refresca la página en tu navegador:

```

deivi@debian:~/infra-test-bonus$ docker-compose logs -f
Attaching to proxy, app
app
  * Serving Flask app 'app'
app
  * Debug mode: off
app
  WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
app
  * Running on all addresses (0.0.0.0)
app
  * Running on http://127.0.0.1:5000
app
  * Running on http://172.20.0.2:5000
app
  Press CTRL+C to quit
app
127.0.0.1 - - [13/Apr/2026 03:19:39] "GET / HTTP/1.1" 200 -
app
127.0.0.1 - - [13/Apr/2026 03:20:09] "GET / HTTP/1.1" 200 -
app
127.0.0.1 - - [13/Apr/2026 03:20:39] "GET / HTTP/1.1" 200 -
proxy
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
proxy
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
proxy
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
proxy
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
proxy
10-listen-on-ipv6-by-default.sh: info: /etc/nginx/conf.d/default.conf differs from the packaged version
proxy
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
proxy
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
proxy
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
proxy
/docker-entrypoint.sh: Configuration complete; ready for start up

```

- **Qué buscar:** los logs de Flask (gracias al `PYTHONUNBUFFERED` que pusimos antes) y los de Nginx combinados con colores diferentes.

2. Ver logs en el disco (SysAdmin):

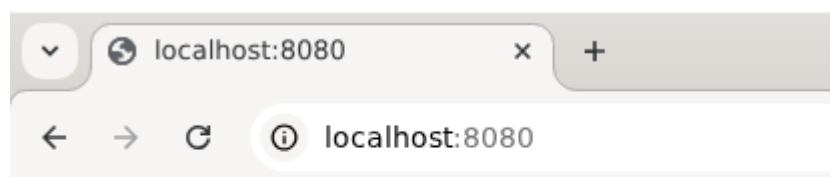
Verifica que Nginx está escribiendo en tu carpeta local de Debian:

- **Qué buscar:** Deberías ver los archivos `access.log` y `error.log`.
- **Prueba:** Ejecuta `tail -f ~/infra-test/logs/nginx/access.log` y verás cómo se registra cada clic que haces en la web.

```

deivi@debian:~/infra-test-bonus$
deivi@debian:~/infra-test-bonus$ ls -l logs/nginx/
total 4
-rw-r--r-- 1 root root 0 abr 12 22:19 access.log
-rw-r--r-- 1 root root 494 abr 12 22:19 error.log
deivi@debian:~/infra-test-bonus$ tail -f logs/nginx/access.log
172.20.0.1 - - [13/Apr/2026:03:22:20 +0000] "GET / HTTP/1.1" 200 22 "-" "Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36" "-"

```



Hello from production!

*"Implementé una estrategia de observabilidad basada en la infraestructura: configuré **log rotation** para prevenir el agotamiento del almacenamiento y utilicé **volúmenes persistentes** para los logs de Nginx, permitiendo que las herramientas de auditoría externas puedan acceder a los datos sin depender del ciclo de vida del contenedor."*

Docker Swarm y Traefik

Paso 1: Activar el modo Swarm

Por defecto, Docker viene en modo "Single Node". Para usar orquestación profesional, debemos activar el modo Clúster.

Comando en VM Debian:

- **Qué hace:** Convierte tu máquina en un "Manager" (Líder) de un clúster. Ahora puedes desplegar "Servicios" en lugar de simples contenedores.

```
deivi@debian:~/infra-test-bonus$ docker swarm init
Swarm initialized: current node (ynylt37zft7vr59jnp2qokstg) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-0gvvck85h8g1oq57m2bcy7su645os70p1ljx4cusu5cz7tmga2-b58v9c0m11ce55cyr0vwj21i8 200.10.20.4:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
```

```
docker swarm join --token SWMTKN-1-0gvvck85h8g1oq57m2bcy7su645os70p1ljx4cusu5cz7tmga2-b58v9c0m11ce55cyr0vwj21i8 200.10.20.4:2377
```

Paso 2: El nuevo archivo `docker-compose.yml` (para Swarm)

En Swarm, Nginx ya no es necesario porque usaremos **Traefik**, que es más potente. Vamos a actualizar tu archivo para que Traefik detecte tu app automáticamente.

Copia este nuevo contenido en tu `docker-compose.yml`:

```
deivi@debian:~$
deivi@debian:~$ cp -r infra-test-bonus/ infra-test-swarm_Traefik
deivi@debian:~$ cd infra-test-swarm_Traefik/
```

```
deivi@debian:~/infra-test-swarm_Traefik$
deivi@debian:~/infra-test-swarm_Traefik$ ls
app  docker-compose.yml  logs  nginx  README.md
deivi@debian:~/infra-test-swarm_Traefik$ nano docker-compose.yml
```

```

GNU nano 7.2                                     docker-compose.yml
#           max-file: "3"           # Solo guardará los últimos 3 archivos

version: '3.8'

services:
  # REEMPLAZO DE NGINX POR TRAEFIK
  traefik:
    image: traefik:v2.10
    command:
      - "--api.insecure=true"           # Panel de control
      - "--providers.docker=true"       # Escuchar a Docker
      - "--providers.docker.swarmMode=true" # Modo Swarm activado
      - "--entrypoints.web.address=:80"  # Puerto de entrada
    ports:
      - "80:80"           # Tráfico público (Sustituye al 8080)
      - "8080:8080"        # Dashboard de Traefik
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro # El "oído" de Traefik
    networks:
      - academia-net
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"
    deploy:
      labels:
        - "traefik.enable=false" # NUEVO: Esto evita el error "port is missing"

  # TU APP FLASK (AHORA COMO SERVICIO ESCALABLE)
  backend:
    image: my-flask-app:latest
    build: ./app
    environment:
      - ENV=production
    networks:
      - academia-net
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:5000/"]
      interval: 30s
      timeout: 10s
      retries: 3
    # CONFIGURACIÓN ESPECÍFICA DE SWARM (REEMPLAZA A CONTAINER_NAME Y DEPENDS_ON)
    deploy:
      replicas: 3
      restart_policy:
        condition: on-failure
      labels:
        - "traefik.enable=true"
        - "traefik.http.routers.backend.rule=Host(`localhost`) || Host(`127.0.0.1`)"
        - "traefik.http.services.backend.loadbalancer.server.port=5000"
        - "traefik.docker.network=mi-academia_academia-net" # NUEVO: Indica la red exacta
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"

# DEFINICIÓN DE LA RED OVERLAY
networks:
  academia-net:
    driver: overlay

```

Paso 3: Desplegar el "Stack"

En Swarm no usamos `up`. Usamos `stack deploy`.

Comandos en tu VM Debian:

1. Construye la imagen primero:

```

deivi@debian:~/infra-test-swarm_Traefik$ docker build -t my-flask-app:latest ./app
Sending build context to Docker daemon 4.608kB
Step 1/12 : FROM python:3.10-slim
--> 94717d9fb8a0
Step 2/12 : RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*
--> Using cache
--> 6438b920efeb
Step 3/12 : ENV PYTHONDONTWRITEBYTECODE=1
--> Using cache
--> 26c237c74473
Step 4/12 : ENV PYTHONUNBUFFERED=1
--> Using cache
--> de938a75f052
Step 5/12 : WORKDIR /app
--> Using cache
--> 47cbf944879b
Step 6/12 : RUN adduser --disabled-password --gecos "" appuser
--> Using cache
--> 357f3cec824a
Step 7/12 : COPY requirements.txt .
--> Using cache
--> 6be9a7fdbb32
Step 8/12 : RUN pip install --no-cache-dir -r requirements.txt
--> Using cache
--> f223dcf4d906
Step 9/12 : COPY . .
--> Using cache
--> bfe8e889e5b3
Step 10/12 : RUN chown -R appuser:appuser /app
--> Using cache
--> d09610bb077d
Step 11/12 : USER appuser
--> Using cache
--> c4cf0c7c1158
Step 12/12 : CMD ["python", "app.py"]
--> Using cache
--> c8af868dd611
Successfully built c8af868dd611
Successfully tagged my-flask-app:latest

```

1. Despliega todo el sistema:

```

deivi@debian:~/infra-test-swarm_Traefik$ docker stack deploy -c docker-compose.yml mi-academia
Ignoring unsupported options: build

Creating network mi-academia_default
Creating service mi-academia_traefik
Creating service mi-academia_web
deivi@debian:~/infra-test-swarm_Traefik$

```

Paso 4: Validación (Cómo ver que funciona)

1. Ver los servicios:

- **Qué buscar:** Verás que el servicio `web` tiene `3/3` réplicas. Si una falla, Swarm levantará otra automáticamente.


```

deivi@debian:~/infra-test-swarm_Traefik$ docker service ls
ID NAME MODE REPLICAS IMAGE PORTS
dv6kvtvxf2ug mi-academia_backend replicated 3/3 my-flask-app:latest
2qt7yy33935x mi-academia_traefik replicated 1/1 traefik:v2.10 *:80->80/tcp, *:8080->8080/tcp
z0vbbjpjthxv mi-academia_web replicated 3/3 my-flask-app:latest

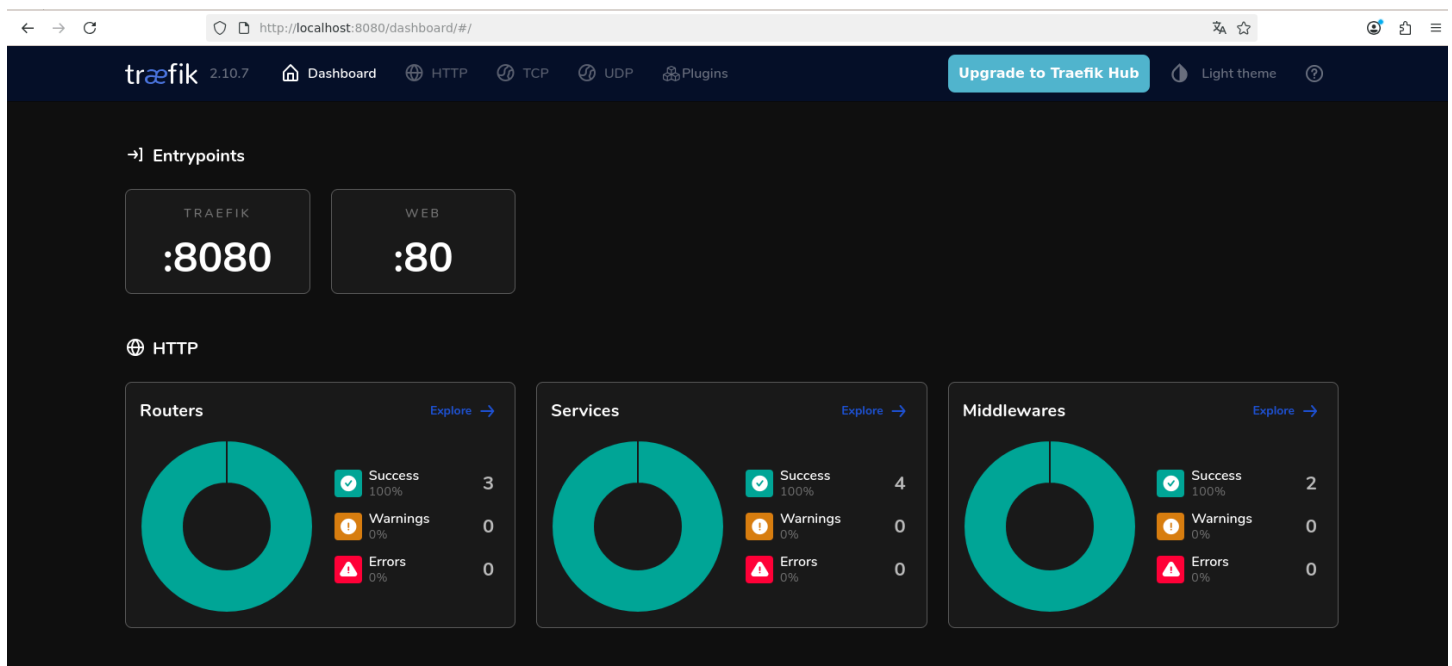
deivi@debian:~/infra-test-swarm_Traefik$ docker service ps mi-academia_backend
NAME IMAGE NODE DESIRED STATE CURRENT STATE ERROR PORTS
ctxe2n69rqp4 mi-academia_backend.1 my-flask-app:latest debian Running Starting 17 seconds ago
x3gs7qwfdbr mi-academia_backend.2 my-flask-app:latest debian Running Starting 17 seconds ago
m72jani3g7my mi-academia_backend.3 my-flask-app:latest debian Running Starting 17 seconds ago

deivi@debian:~/infra-test-swarm_Traefik$ docker service ps mi-academia_traefik
NAME IMAGE NODE DESIRED STATE CURRENT STATE ERROR PORTS
ctxe2n69rqp4 mi-academia_backend.1 my-flask-app:latest debian Running Running less than a second ago
x3gs7qwfdbr mi-academia_backend.2 my-flask-app:latest debian Running Running 1 second ago
m72jani3g7my mi-academia_backend.3 my-flask-app:latest debian Running Running 1 second ago

```

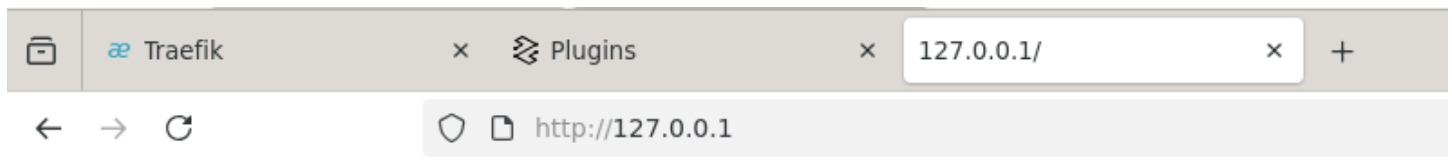
2. Ver el Panel de Traefik:

Entra en tu navegador a <http://localhost:8080>. Verás una interfaz gráfica que muestra cómo Traefik encontró tu app Flask y le creó una ruta automáticamente.



3. Probar la App:

Entra a <http://localhost>. Traefik recibirá la petición y la enviará a una de las 3 réplicas de forma balanceada.



Hello from production!

Resumen

Se ha pasado de un servidor estático a una **arquitectura elástica**:

1. **Alta Disponibilidad**: Tienes 3 réplicas. Si borras una (`docker rm`), Swarm la recrea al instante.

```
Hello from production!deivi@debian:~/infra-test-swarm/Traefik$ docker service logs mi-academia_backend
mi-academia_backend.2.x3gs7qwfdbir@debian: * Serving Flask app 'app'
mi-academia_backend.2.x3gs7qwfdbir@debian: * Debug mode: off
mi-academia_backend.2.x3gs7qwfdbir@debian: WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
mi-academia_backend.2.x3gs7qwfdbir@debian: * Running on all addresses (0.0.0.0)
mi-academia_backend.2.x3gs7qwfdbir@debian: * Running on http://127.0.0.1:5000
mi-academia_backend.2.x3gs7qwfdbir@debian: * Running on http://172.21.0.4:5000
mi-academia_backend.2.x3gs7qwfdbir@debian: Press CTRL+C to quit
127.0.0.1 - - [14/Apr/2026 00:48:50] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:49:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:49:50] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:50:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:50:50] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:51:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:51:51] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:52:21] "GET / HTTP/1.1" 200 -
10.0.3.3 - - [14/Apr/2026 00:52:15] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:52:21] "GET / HTTP/1.1" 200 -
mi-academia_backend.1.ctxe2n69rqp4@debian: * Serving Flask app 'app'
mi-academia_backend.1.ctxe2n69rqp4@debian: * Debug mode: off
mi-academia_backend.1.ctxe2n69rqp4@debian: WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
mi-academia_backend.1.ctxe2n69rqp4@debian: * Running on all addresses (0.0.0.0)
mi-academia_backend.1.ctxe2n69rqp4@debian: * Running on http://127.0.0.1:5000
mi-academia_backend.1.ctxe2n69rqp4@debian: * Running on http://172.21.0.6:5000
mi-academia_backend.1.ctxe2n69rqp4@debian: Press CTRL+C to quit
127.0.0.1 - - [14/Apr/2026 00:48:50] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:49:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:49:50] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:50:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:50:50] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:51:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:51:51] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:52:21] "GET / HTTP/1.1" 200 -
10.0.3.3 - - [14/Apr/2026 00:52:42] "GET / HTTP/1.1" 200 -
mi-academia_backend.3.m72jan13g7my@debian: * Serving Flask app 'app'
mi-academia_backend.3.m72jan13g7my@debian: * Debug mode: off
mi-academia_backend.3.m72jan13g7my@debian: WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
mi-academia_backend.3.m72jan13g7my@debian: * Running on all addresses (0.0.0.0)
mi-academia_backend.3.m72jan13g7my@debian: * Running on http://127.0.0.1:5000
mi-academia_backend.3.m72jan13g7my@debian: * Running on http://172.21.0.5:5000
mi-academia_backend.3.m72jan13g7my@debian: Press CTRL+C to quit
127.0.0.1 - - [14/Apr/2026 00:48:50] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:49:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:49:50] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:50:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:50:50] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:51:20] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:51:51] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [14/Apr/2026 00:52:21] "GET / HTTP/1.1" 200 -
10.0.3.3 - - [14/Apr/2026 00:51:44] "GET / HTTP/1.1" 200 -
```

2. **Service Discovery**: No se tuvo que configurar la IP de la app en Traefik; él la encontró por "etiquetas" (labels).
3. **Zero Downtime**: Podrías actualizar la app y Swarm iría cambiando los contenedores uno por uno sin que el servicio caiga.
4. **Simulación de Caída**

Console 1 (Monitoring)

```
deivi@debian:~/infra-test-swarm_Traefik$ watch docker service ps mi-academia_backend
deivi@debian:~/infra-test-swarm_Traefik$ watch docker service ps mi-academia_backend
deivi@debian:~/infra-test-swarm_Traefik$ watch docker service ps mi-academia_backend
deivi@debian:~/infra-test-swarm_Traefik$ █
```

```
Cada 2,0s: docker service ps mi-academia_backend
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
vyg9r8zh9lr0	mi-academia_backend.1	my-flask-app:latest	debian	Running	Starting 12 seconds ago		
ctxe2n69rqp4	_mi-academia_backend.1	my-flask-app:latest	debian	Shutdown	Failed 17 seconds ago	"task: non-zero exit (137)"	
x3gs7qwfdbir	mi-academia_backend.2	my-flask-app:latest	debian	Running	Running about an hour ago		
m72jani3g7my	mi-academia_backend.3	my-flask-app:latest	debian	Running	Running about an hour ago		

Console 2 (Matar proceso de una replica)

```
deivi@debian:~/infra-test-swarm_Traefik$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
43af9f3edb19	my-flask-app:latest	"python app.py"	About an hour ago	Up About an hour (healthy)		mi-academia_backend.1.ctxe2n69rqp46dfbt9459eaii
7378f83789fc	my-flask-app:latest	"python app.py"	About an hour ago	Up About an hour (healthy)		mi-academia_backend.2.x3gs7qwfdbirguu3fd418c145
984d9c6414a3	my-flask-app:latest	"python app.py"	About an hour ago	Up About an hour (healthy)		mi-academia_backend.3.m72jani3g7mymcaio3sq2lop
b31d1b82d5ae	traefik:v2.10	"/entrypoint.sh --ap..."	About an hour ago	Up About an hour	80/tcp	mi-academia_traefik.1.4iuz04fnwcupvxkpad6nr4bj5

```
deivi@debian:~/infra-test-swarm_Traefik$ docker rm -f 43af9f3edb19
43af9f3edb19
deivi@debian:~/infra-test-swarm_Traefik$ █
```

5. Aumentar el numero de replicas de 3 a 5

```
deivi@debian:~/infra-test-swarm_Traefik$ docker service scale mi-academia_backend=5
mi-academia_backend scaled to 5
overall progress: 3 out of 5 tasks
1/5: running [=====>]
2/5: starting [=====>]
3/5: running [=====>]
4/5: running [=====>]
5/5: starting [=====>]
█
```

```
deivi@debian:~/infra-test-swarm_Traefik$
deivi@debian:~/infra-test-swarm_Traefik$ docker service ls
```

ID	NAME	MODE	REPLICAS	IMAGE	PORTS
tso5qenx7ki2	mi-academia_backend	replicated	5/5	my-flask-app:latest	
5y5ja1bnqn4t	mi-academia_traefik	replicated	1/1	traefik:v2.10	*:80->80/tcp, *:8080->8080/tcp

```
deivi@debian:~/infra-test-swarm_Traefik$ █
```

```
deivi@debian:~/infra-test-swarm_Traefik$ docker service ps mi-academia_backend
```

ID	NAME	IMAGE	NODE	DESIRED STATE	CURRENT STATE	ERROR	PORTS
vyg9r8zh9lr0	mi-academia_backend.1	my-flask-app:latest	debian	Running	Running 8 minutes ago		
ctxe2n69rqp4	_mi-academia_backend.1	my-flask-app:latest	debian	Shutdown	Failed 8 minutes ago	"task: non-zero exit (137)"	
x3gs7qwfdbir	mi-academia_backend.2	my-flask-app:latest	debian	Running	Running about an hour ago		
m72jani3g7my	mi-academia_backend.3	my-flask-app:latest	debian	Running	Running about an hour ago		
jnick1s25t5j	mi-academia_backend.4	my-flask-app:latest	debian	Running	Running 48 seconds ago		
svcgtd578eq9	mi-academia_backend.5	my-flask-app:latest	debian	Running	Running 48 seconds ago		

```
deivi@debian:~/infra-test-swarm_Traefik$ █
```

The screenshot shows the Traefik 2.10.7 dashboard. The top navigation bar includes links for Dashboard, HTTP, TCP, UDP, and Plugins. Below this, there are tabs for HTTP Routers (3), HTTP Services (4), and HTTP Middlewares (2). The main content area is titled 'backend@docker'. It is divided into two sections: 'Service Details' and 'Servers'. The 'Service Details' section shows the type as 'loadbalancer', the provider as 'Docker', the status as 'Success', and the 'PASS HOST HEADER' option as 'True'. The 'Servers' section is highlighted with a red border and contains a table with two columns: 'Status' and 'URL'. The table lists five servers, all with a status of 'Success' and various IP addresses on port 5000.

Status	URL
✓	http://10.0.3.11:5000
✓	http://10.0.3.8:5000
✓	http://10.0.3.10:5000
✓	http://10.0.3.9:5000
✓	http://10.0.3.7:5000

Integración Continua (CI)

Integración Continua (CI). Es el "robot" que valida que tu código sea perfecto antes de que llegue a los servidores de la academia.

Vamos a hacerlo desde cero, paso a paso.

Paso 1: Crear cuenta y repositorio en GitHub

Si ya tienes cuenta, salta al punto 2. Si no:

1. Ve a github.com y regístrate.
2. Una vez dentro, haz clic en el botón verde **"New"** (o el símbolo **+**) para crear un repositorio.
3. Nombre: `infra-test-academia`.
4. Déjalo como **Public** (para que el reclutador lo vea) y dale a **"Create repository"**.

Create repository Import repository

 You've been added to the **susito** organization. [View organization](#) ✕

 Trending repositories · [See more](#)

 microsoft/markdown

Python tool for converting files and office documents to Markdown.

 Python

 99.3k

 coleam00/Archon

The first open-source harness builder for AI coding. Make AI coding deterministic and repeatable.

 TypeScript

 15.4k

- 33 minutes ago
Actions workflows are limited to 50 reruns
- 33 minutes ago
Copilot CLI activity now included in usage metrics totals and feature...
- 8 hours ago
Copilot cloud agent's validation tools are now 20% faster
- Yesterday
Ask Copilot in security assessments now available

[View changelog](#) ...

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

1 General

Owner *

 deivicubillos ▾

Repository name *

infra-test-academia

✔ infra-test-academia is available.

Great repository names are short and memorable. How about [fluffy-train](#)?

Description

infra-test-academia

19 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

 Public ▾

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off ☐

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore ▾

Add license

Licenses explain how others can use your code. [About licenses](#)

No license ▾

Create repository

<https://github.com/deivicubillos/infra-test-academia.git>

Paso 2: Inicializar Git en tu VM Debian

Ahora vamos a conectar tu carpeta local con GitHub. Ejecuta estos comandos dentro de tu carpeta `infra-test-swarm_Traefik`:

```
# 1. Inicializa el repositorio local
```

```
git init
```

2. Configura tu identidad (usa tu correo de GitHub)

```
git config --global user.email "tu@email.com"
```

```
git config --global user.name "Tu Nombre"
```

3. Añade todos los archivos

```
git add .
```

4. Crea tu primer commit

```
git commit -m "Initial commit: Flask + Swarm + Traefik"
```

5. Cambia la rama a main

```
git branch -M main
```

6. Conecta con tu repo de GitHub (Copia el link de tu repo)

```
git remote add origin https://github.com/TU_USUARIO/infra-test-academia.git
```

```
deivi@debian:~/infra-test-swarm_Traefik$ git init
ayuda: Usando 'master' como el nombre de la rama inicial. Este nombre de rama predeterminado
ayuda: está sujeto a cambios. Para configurar el nombre de la rama inicial para usar en todos
ayuda: de sus nuevos repositorios, reprimiendo esta advertencia, llama a:
ayuda:
ayuda: git config --global init.defaultBranch <nombre>
ayuda:
ayuda: Los nombres comúnmente elegidos en lugar de 'master' son 'main', 'trunk' y
ayuda: 'development'. Se puede cambiar el nombre de la rama recién creada mediante este comando:
ayuda:
ayuda: git branch -m <nombre>
Inicializado repositorio Git vacío en /home/deivi/infra-test-swarm_Traefik/.git/
```

```
deivi@debian:~/infra-test-swarm_Traefik$ git config --global user.email "deivicubillos@gmail.com"
deivi@debian:~/infra-test-swarm_Traefik$ git config --global user.name "Deivi Cubillos"
deivi@debian:~/infra-test-swarm_Traefik$ git add .
deivi@debian:~/infra-test-swarm_Traefik$ git commit -m "Initial Commit: Flask + Swarm + Traefik"
[master (commit-raíz) ff30b2a] Initial Commit: Flask + Swarm + Traefik
12 files changed, 234 insertions(+)
create mode 100644 ..env.swp
create mode 100644 README.md.swp
create mode 100644 .env
create mode 100644 .github/workflows/deploy.yml
create mode 100644 README.md
create mode 100644 app/Dockerfile
create mode 100644 app/app.py
create mode 100644 app/requirements.txt
create mode 100644 docker-compose.yml
create mode 100644 logs/nginx/access.log
create mode 100644 logs/nginx/error.log
create mode 100644 nginx/default.conf
deivi@debian:~/infra-test-swarm_Traefik$
```

```
deivi@debian:~/infra-test-swarm_Traefik$
deivi@debian:~/infra-test-swarm_Traefik$ git branch -M main
deivi@debian:~/infra-test-swarm_Traefik$
deivi@debian:~/infra-test-swarm_Traefik$ git remote add origin https://github.com/deivicubillos/infra-test-academia.git
deivi@debian:~/infra-test-swarm_Traefik$
```

Paso 3: Crear el archivo de Integración Continua (CI)

El "robot" de GitHub se configura en una carpeta específica. Vamos a crearla:

1. Crea las carpetas:

2. Crea el archivo del pipeline:

```
deivi@debian:~/infra-test-swarm_Traefik$ mkdir -p .github/workflows/  
deivi@debian:~/infra-test-swarm_Traefik$ nano .github/workflows/deploy.yml
```

3. Pega este contenido:

```
GNU nano 7.2 .github/workflows/deploy.yml  
  
name: CI Academia Pipeline  
  
# El robot se activa cada vez que subes código (push) a la rama main  
on:  
  push:  
    branches: [ main ]  
  
jobs:  
  build-and-test:  
    runs-on: ubuntu-latest # El robot usa un Ubuntu limpio en la nube  
  
    steps:  
      - name: 1. Descargar código  
        uses: actions/checkout@v3  
  
      - name: 2. Validar Dockerfile (Linting)  
        run: |  
          # Esto verifica que el Dockerfile no tenga errores de sintaxis  
          docker build -t test-image ./app  
  
      - name: 3. Escaneo de Seguridad (Trivy)  
        uses: aquasecurity/trivy-action@master  
        with:  
          image-ref: 'test-image'  
          format: 'table'  
          severity: 'CRITICAL,HIGH'  
  
      - name: 4. Prueba de funcionamiento (Unit Test)  
        run: |  
          # Levantamos un contenedor temporal para ver si responde  
          docker run -d --name test-app -p 5000:5000 test-image  
          sleep 5  
          curl --fail http://localhost:5000 || exit 1  
          docker rm -f test-app  
  
      - name: 5. Validar Orquestación (Compose)  
        run: |  
          # Verifica que el archivo de Swarm sea válido  
          docker compose config
```

```
deivi@debian:~/infra-test-swarm_Traefik$  
deivi@debian:~/infra-test-swarm_Traefik$ git add .  
deivi@debian:~/infra-test-swarm_Traefik$ git commit -m "Add CI pipeline"  
[main 74f5c63] Add CI pipeline  
1 file changed, 53 insertions(+), 12 deletions(-)
```



```
deivi@debian:~/infra-test-swarm_Traefik$  
deivi@debian:~/infra-test-swarm_Traefik$ git push -u origin main  
Username for 'https://github.com': deivicubillos@gmail.com  
Password for 'https://deivicubillos%40gmail.com@github.com':  
Enumerando objetos: 25, listo.  
Contando objetos: 100% (25/25), listo.  
Compresión delta usando hasta 2 hilos  
Comprimiendo objetos: 100% (17/17), listo.  
Escribiendo objetos: 100% (25/25), 4.82 KiB | 822.00 KiB/s, listo.  
Total 25 (delta 3), reusados 0 (delta 0), pack-reusados 0  
remote: Resolving deltas: 100% (3/3), done.  
To https://github.com/deivicubillos/infra-test-academia.git  
* [new branch]      main -> main  
rama 'main' configurada para rastrear 'origin/main'.  
deivi@debian:~/infra-test-swarm_Traefik$
```

Nota: Te pedirá tu usuario y un "Personal Access Token" (GitHub ya no usa contraseñas simples por terminal. Si no sabes crear el token, dime y te explico).

Generar el Token en GitHub

1. Haz clic en tu **foto de perfil** (esquina superior derecha) y ve a **Settings**.
2. Baja hasta el final en el menú de la izquierda y haz clic en **Developer settings**.
3. Selecciona **Personal access tokens** -> **Tokens (classic)**.
4. Haz clic en el botón **Generate new token** -> **Generate new token (classic)**.
5. **Configuración:**
 - **Note:** Ponle un nombre, por ejemplo: **Debian-VM-Lab**.
 - **Expiration:** Puedes ponerle 30 días o "No expiration" (solo para este lab).
 - **Select scopes:** Selecciona la casilla **repo** (esto permite subir código). Es la única que necesitas por ahora. también seleccionar la opción **workflows**
6. Baja al final y haz clic en **Generate token**.

 **IMPORTANTE:** Copia el token que aparece (empieza por **ghp_...**).

Guárdalo en un bloc de notas, porque una vez que cierres la página, no podrás volver a verlo.

Settings / Developer Settings

Search

GitHub Apps

OAuth Apps

Personal access tokens

Fine-grained tokens

Tokens (classic)

New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Note

Debian-VM-Lab

What's this token for?

Expiration

No expiration

GitHub strongly recommends that you set an expiration date for your token to help keep your information secure. [Learn more](#)

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes](#).

☒ repo	Full control of private repositories
☒ repo:status	Access commit status
☒ repo_deployment	Access deployment status
☒ public_repo	Access public repositories
☒ repo:invite	Access repository invitations
☒ security_events	Read and write security events
☐ workflow	Update GitHub Action workflows

☒ repo

Full control of private repositories

☒ repo:status

Access commit status

☒ repo_deployment

Access deployment status

☒ public_repo

Access public repositories

☒ repo:invite

Access repository invitations

☒ security_events

Read and write security events

☒ workflow

Update GitHub Action workflows

Settings / Developer Settings

Search

Some of the scopes you've selected are included in other scopes. Only the minimum set of necessary scopes has been saved.

GitHub Apps

OAuth Apps

Personal access tokens

Fine-grained tokens

Tokens (classic)

Personal access tokens (classic)

Generate new token

Tokens you have generated that can be used to access the [GitHub API](#).

Make sure to copy your personal access token now. You won't be able to see it again!

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

Delete

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Token: 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

Paso 5: Ver el robot en acción (Validación)

1. Entra a tu repositorio en la web de GitHub.
2. Haz clic en la pestaña **"Actions"**.

- Verás un flujo llamado **"CI Academia Pipeline"** con un círculo amarillo (procesando) o verde (éxito).
- Haz clic en él para ver cómo el robot construye la imagen, la escanea en busca de virus y prueba que el `curl` funcione.

deivucubillos / infra-test-academia

Code Issues Pull requests **Actions** Projects Wiki Security and quality Insights Settings

Actions [New workflow](#)

All workflows

CI Academia Pipeline

Management

- Caches
- Attestations
- Runners
- Usage metrics
- Performance metrics

All workflows

Showing runs from all workflows

Filter workflow runs

Help us improve GitHub Actions

Tell us how to make GitHub Actions work better for you with three quick questions.

Give feedback

1 workflow run

Workflow Event Status Branch Actor

Add CI pipeline

CI Academia Pipeline #1: Commit 74f5c63 pushed by deivucubillos

main

3 minutes ago

40s

deivucubillos / infra-test-academia

Code Issues Pull requests **Actions** Projects Wiki Security and quality Insights Settings

CI Academia Pipeline

Add CI pipeline #1

Re-run all jobs

Summary

All jobs

build-and-test

Run details

Usage

Workflow file

Triggered via push 4 minutes ago

deivucubillos pushed 74f5c63 main

Status **Success**

Total duration **40s**

Artifacts --

deploy.yml

on: push

build-and-test 37s

Annotations

1 warning

build-and-test

Node.js 20 actions are deprecated. The following actions are running on Node.js 20 and may not work as expected: actions/cache@40085f644dc79613175e3cd80713209898809, actions/checkout@v3. Actions will be forced to run with Node.js 24 by default sta.

Show more

github.com/deivucubillos/infra-test-academia/actions/runs/24378076782/job/71195588559

Code Issues Pull requests **Actions** Projects Wiki Security and quality Insights Settings

CI Academia Pipeline

Add CI pipeline #1

Re-run all jobs

Summary

All jobs

build-and-test

Run details

Usage

Workflow file

Annotations

1 warning

build-and-test

succeeded 3 minutes ago in 37s

Search logs

- Set up job 3s
- 1. Descargar código 0s
- 2. Validar Dockerfile (Linting) 11s
- 3. Escaneo de Seguridad (Trivy) 11s
- 4. Prueba de funcionamiento (Unit Test) 5s
- 5. Validar Orquestación (Compose) 0s
- Post 3. Escaneo de Seguridad (Trivy) 5s
- Post 1. Descargar código 0s
- Complete job 0s

"Se implemento un flujo de **Integración Continua (CI) usando GitHub Actions. Cada vez que se sube código, un pipeline automatizado realiza tres validaciones críticas:*

- 1. **Build Check**: Asegura que la imagen de Docker se construya correctamente.*
- 2. **Security Scanning**: Uso Trivy para detectar vulnerabilidades críticas en las librerías.*
- 3. **Integration Test**: Levanto un contenedor efímero y realizo un healthcheck real con `curl` para garantizar que la aplicación responde antes de considerar el código como válido."**